Projet d'implantation de réseau

Conception et mise en place d'une solution SIEM dans un SOC automatisé

Surveillance réseau et automatisation

Comprendre comment un réseau est construit, protégé, surveillé, et comment les équipes de sécurité détectent, analysent et répondent à une attaque.

Plan et fil conducteur

> On avance du général au particulier : l'entreprise, la sécurité, le SOC, le SIEM, les outils, puis les humains qui s'en servent.

1. L'entreprise et sa sécurité

Architecture réseau, zones, défense en profondeur, concepts de sécurité, pare-feu, IDS/IPS, switch et port SPAN.

2. Le SOC

Définition, objectifs (MTTD, MTTR), niveaux d'analystes, rôles, technologies (SIEM, EDR, NDR, XDR, SOAR).

3. Les données et le SIEM

Logs, événements, collecte, composants du SIEM, pipeline interne, détection, alertes et leur qualité.

4. Les outils open source

Wazuh, Security Onion, Suricata, Zeek, Wireshark, TheHive, Cortex, MISP, Shuffle, OpenVAS : rôle de chacun.

5. L'humain et l'automatisation

Chronologie d'une attaque, travail des analystes, réponse à incident, SOAR, IA, amélioration continue.

Glossaire

Les termes techniques, définis un par un.

Un seul exemple pour tout le projet

Un attaquant (adresse 192.168.163.136, par exemple une machine Kali Linux) tente de deviner le mot de passe SSH d'un serveur (adresse 192.168.163.142). À chaque notion, nous verrons ce qui arrive à **cette** attaque : quelle trace elle laisse, qui la voit, qui la traite.

Comment lire chaque notion

On répond toujours aux mêmes questions : de quoi s'agit-il, pourquoi cela existe, comment cela fonctionne, qui intervient et ce que cela produit.

Partie 1

1 L'entreprise et sa sécurité

1.1 L'architecture d'un réseau d'entreprise

Avant de surveiller un réseau, il faut savoir comment il est construit. La plupart des entreprises suivent le **modèle hiérarchique à trois couches**, qui sépare les rôles pour que le réseau reste fiable, extensible et facile à superviser.

Le cœur (core) relie les grands blocs du réseau à très haut débit. Sa seule mission est de transporter vite et sans interruption : on évite d'y mettre du filtrage lourd.

La distribution regroupe les switchs d'accès. Elle relie les VLAN entre eux (un VLAN est un réseau local virtuel : une découpe logique d'un switch en plusieurs réseaux isolés) et applique les politiques : listes de contrôle d'accès (ACL), qualité de service. C'est le point de contrôle naturel.

L'accès est l'endroit où se branchent postes, téléphones IP, imprimantes et bornes Wi-Fi. On y applique la sécurité de port : authentification 802.1X, limitation des adresses MAC, affectation au bon VLAN.

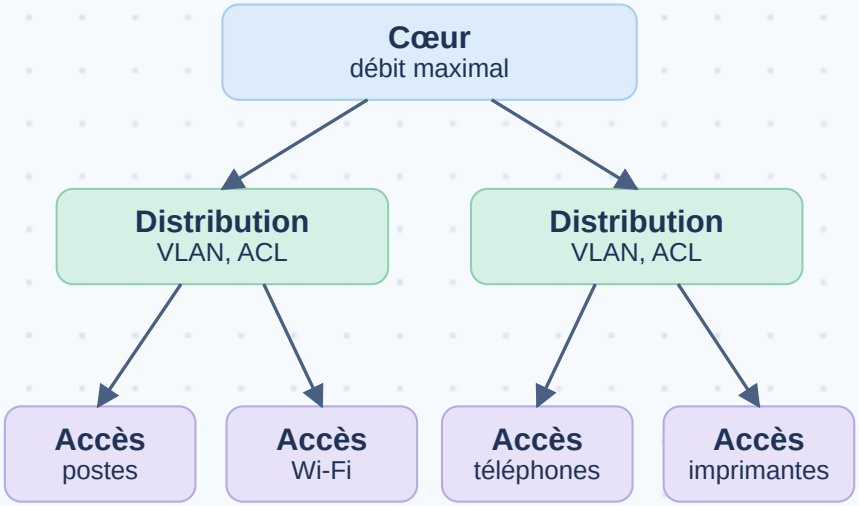


Figure 1 : *Les trois couches : plus on est proche de l'accès, plus on voit les utilisateurs.*

Pour la sécurité

Dans une petite entreprise, cœur et distribution sont souvent fusionnés. Pour un SOC, cette hiérarchie indique où placer les capteurs : au cœur on voit beaucoup de trafic agrégé, à l'accès on voit chaque poste.

1.2 Segmenter : zones et pare-feu

Un réseau sans cloisons est un réseau où l'attaquant va partout dès qu'il est entré. On le découpe donc en **zones** de confiance différente, séparées par un pare-feu.

La **DMZ** (zone démilitarisée) héberge les services visibles depuis Internet : site web, relais de messagerie. Si l'un est compromis, l'attaquant reste dans la DMZ et n'accède pas directement au réseau interne.

Le **LAN utilisateurs** regroupe les postes. La **zone des serveurs internes** contient bases de données et annuaire. Un **réseau de gestion** séparé sert à administrer les équipements.

Entre les zones, la règle est « tout est interdit sauf ce qui est nécessaire » (principe du moindre privilège). Cette segmentation freine le **mouvement latéral** : le déplacement d'un attaquant d'une machine à une autre pour atteindre sa vraie cible.

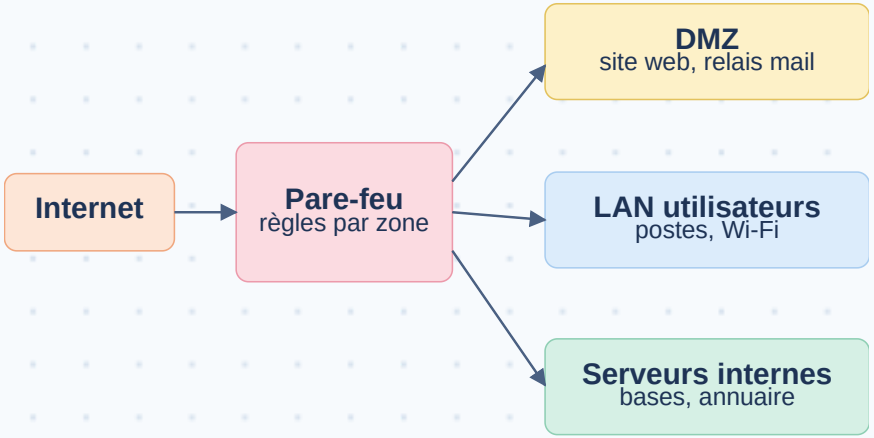


Figure 2 : *Un pare-feu au centre : chaque zone a ses propres règles.*

Conséquence pour la surveillance

Chaque passage entre deux zones est une occasion d'observer. Le pare-feu produit des journaux de connexions acceptées ou refusées, très utiles pour reconstituer le chemin d'un attaquant.

1.3 La défense en profondeur : la sécurité en oignon

Aucune protection n'est infaillible. La **défense en profondeur** consiste à empiler plusieurs couches indépendantes, de sorte que la défaillance de l'une soit rattrapée par la suivante. L'attaquant doit franchir chaque couche, et chaque franchissement laisse une trace.

De l'extérieur vers le centre : les **politiques et les humains** (charte, sensibilisation au phishing, gestion des droits) ; le **périmètre** (pare-feu, VPN, filtrage de messagerie) ; le **réseau interne** (segmentation, 802.1X, IDS/IPS) ; l'**hôte** (durcissement, correctifs, antivirus ou EDR, journalisation) ; l'**application** (validation des entrées, authentification forte) ; les **données** (chiffrement, sauvegardes, droits d'accès).

Ces couches servent trois fonctions : **prévenir** (empêcher), **détecter** (voir ce qui passe malgré tout) et **répondre** (limiter et réparer). Le SOC s'occupe des deux dernières, et collecte des traces à chaque couche.

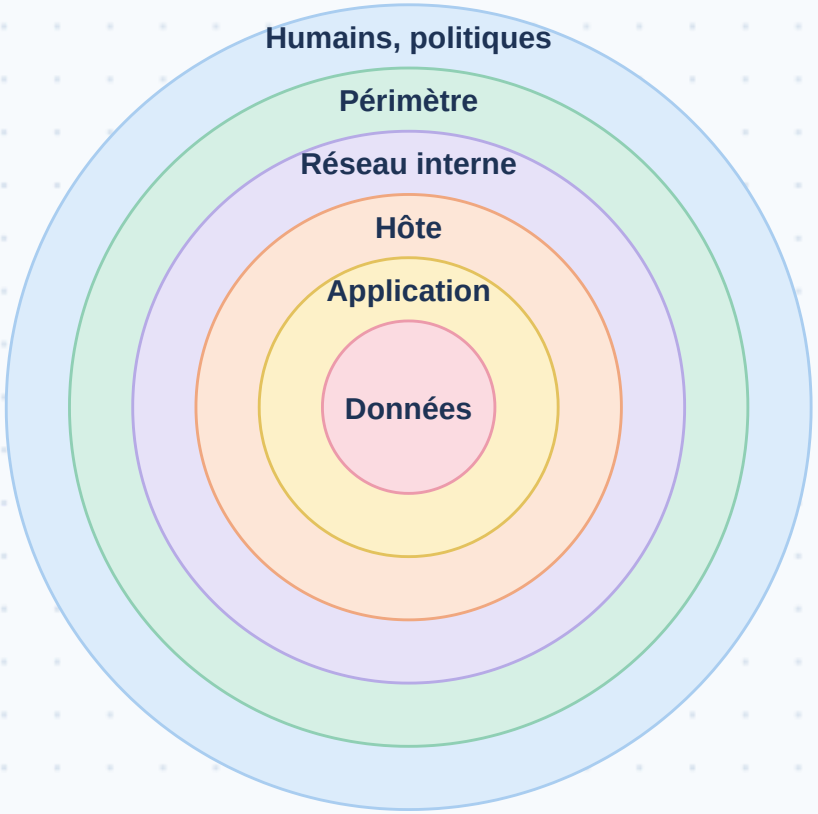


Figure 3 : *L'oignon : l'attaquant doit traverser toutes les couches pour atteindre les données.*

1.4 Les concepts de base de la sécurité

> Un petit vocabulaire précis évite de confondre ce qui existe (une faiblesse), ce qui peut arriver (une menace) et ce qui arrive (une attaque).

Actif
Tout ce qui a de la valeur : donnée, serveur, compte, service.

Vulnérabilité
Faiblesse exploitable : logiciel non corrigé, mot de passe faible, mauvaise configuration.

Menace
Cause potentielle d'un incident : un attaquant, un logiciel malveillant, une erreur interne.

Attaque
Action réelle d'exploitation d'une vulnérabilité par une menace.

Risque
Probabilité qu'une menace exploite une vulnérabilité, multipliée par l'impact sur l'actif.

Surface d'attaque
Ensemble des points par lesquels on peut attaquer : ports ouverts, applications, utilisateurs.

Vecteur d'attaque
Chemin utilisé : e-mail piégé, service exposé, compte volé, clé USB.

Mouvement latéral
Déplacement de machine en machine après la première intrusion.

1.5 La triade CIA

Confidentialité
Seules les personnes autorisées accèdent à l'information : chiffrement, contrôle d'accès, authentification.

Intégrité
L'information n'est pas modifiée sans que cela se voie : hachage (empreinte), signatures numériques.

Disponibilité
Les ressources sont accessibles quand on en a besoin : redondance, sauvegardes, protection contre le déni de service (DoS).

Et la traçabilité ?
Elle ne fait pas partie du triptyque, mais elle permet de prouver et de reconstituer. Sans journaux fiables, impossible de savoir ce qui s'est passé : c'est la matière première du SOC.

1.6 Le pare-feu et ses limites

Un **pare-feu** filtre le trafic entre deux zones selon des règles : adresses source et destination, ports, protocoles. Un pare-feu **à état** (stateful) suit les connexions et n'autorise un retour que s'il répond à une demande légitime. Un pare-feu **nouvelle génération** (NGFW) identifie aussi l'application et l'utilisateur, et inclut parfois un IPS.

Ses limites expliquent pourquoi il faut un SOC : il laisse passer le trafic autorisé même s'il est malveillant (une attaque web sur le port 443, un mot de passe volé via le VPN) ; il ne lit pas le contenu chiffré sans déchiffrement ; il ne voit pas ce qui se passe à l'intérieur d'une machine ; et il ne voit pas le trafic interne qui ne le traverse pas. Dans notre exemple, il voit les tentatives vers le port 22, mais ne sait pas si un mot de passe a fini par fonctionner.

1.7 IDS et IPS

Un **IDS** (Intrusion Detection System) analyse le trafic et **lève une alerte** quand il reconnaît une attaque. Il est *passif* : il observe une copie du trafic et ne peut rien bloquer. Un **IPS** (Intrusion Prevention System) est placé *en ligne* sur le chemin du trafic : il peut le **bloquer** en temps réel, mais une fausse alerte coupe alors une communication légitime.

On distingue le **NIDS** (réseau, qui lit les paquets) du **HIDS** (hôte, qui lit les journaux, les fichiers et les processus d'une machine). La détection repose soit sur des **signatures** (motifs d'attaques connues), soit sur la détection d'**anomalies** (écart par rapport à un comportement normal).

1.8 Le switch et le port SPAN

Un switch ne renvoie une trame qu'au port concerné : un poste branché sur un autre port ne voit donc pas le trafic des autres. Pour surveiller, on demande au switch de **copier** le trafic de certains ports ou VLAN vers un port de destination : c'est le **port mirroring**, appelé SPAN (Switched Port Analyzer) chez Cisco. La copie peut porter sur le trafic entrant, sortant ou les deux ; pour l'analyse, les deux sont préférables.

Le poste de surveillance utilise une **carte de capture** sans adresse IP, en **mode promiscuité** (elle accepte tous les paquets, pas seulement ceux qui lui sont adressés), et une seconde carte pour sa gestion.

Ce que reçoit le poste est une **copie de paquets**, rien de plus : ni alerte, ni log. Il faut un outil de capture et d'analyse (une sonde) pour en tirer des alertes ou des métadonnées.

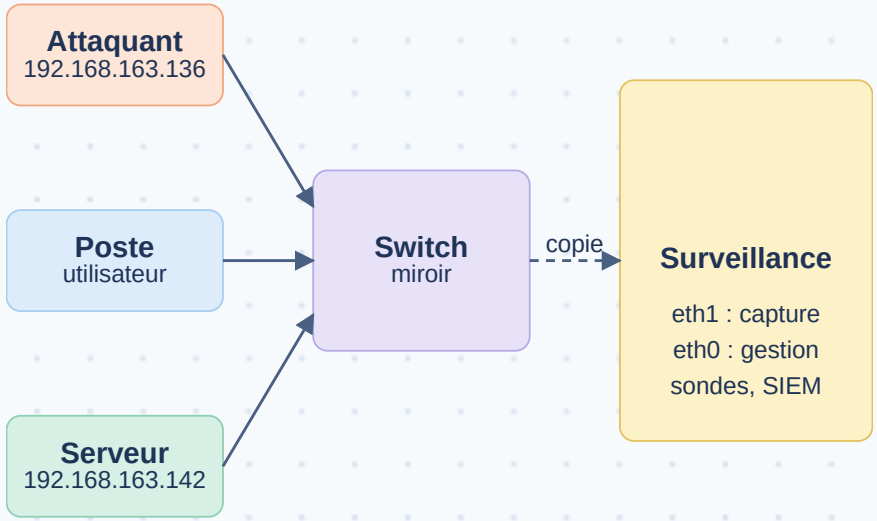


Figure 4 : *Le switch transmet normalement le trafic ; une copie part vers le poste de surveillance.*

Les limites du SPAN

Si la somme des ports copiés dépasse le débit du port de destination, le switch **abandonne des copies**. Le trafic chiffré (TLS) n'est lisible que par ses métadonnées. La sonde ne peut pas bloquer, car elle observe une copie. Un **TAP** (boîtier dédié) évite les pertes mais coûte plus cher : SPAN en laboratoire, TAP en production.

Partie 2

2 Le SOC

2.1 Qu'est-ce qu'un SOC ?

> Le Security Operations Center est l'équipe, avec ses procédures et ses outils, qui surveille la sécurité d'un système d'information en continu.

Sa mission : **réduire le temps pendant lequel une attaque reste invisible ou active**. Pour cela il surveille, détecte, analyse, répond, puis améliore la défense avec ce qu'il a appris.

Un SOC repose sur trois piliers inséparables. Les **personnes** (analystes, ingénieurs, manager), les **processus** (procédures, playbooks, règles d'escalade, gestion des incidents) et les **technologies** (SIEM, EDR, IDS, SOAR). Un SIEM sans équipe ne produit que des tableaux de bord que personne ne lit ; des analystes sans outils cherchent à la main dans des fichiers épars, trop lentement.

Le **périmètre** du SOC est ce qu'il surveille réellement : postes, serveurs, réseau, identités, applications, cloud. Ce qui est hors périmètre n'est pas vu.

Deux indicateurs mesurent son efficacité. Le **MTTD** (Mean Time To Detect) est la moyenne, sur plusieurs incidents, du délai entre le début d'une attaque et sa détection. Le **MTTR** (Mean Time To Respond) est la moyenne du délai entre l'alerte et la réponse. Plus ils sont courts, moins l'attaquant a de temps.

SOC interne

Géré par l'organisation. Contrôle total, coût élevé.

SOC externalisé

Un prestataire (MSSP) surveille pour plusieurs clients. Moins cher, dépendance.

SOC hybride

Supervision interne partielle, externalisation ciblée (nuit, week-end).

SOC distribué

Équipe répartie géographiquement, outils partagés. Adapté au cloud.

2.2 Niveaux d'analystes et rôles

Les alertes sont nombreuses et la plupart sont bénignes. On répartit donc le travail par niveau de difficulté, pour libérer les experts rares pour les incidents qui le méritent. Le dossier passe d'un niveau à l'autre : c'est le **passage de responsabilité**.

Tier 1 : la première ligne

Il surveille la file d'alertes, lit chaque alerte, vérifie le contexte (l'adresse est-elle connue ? le compte est-il légitime ?), écarte les **faux positifs** (fausses alertes) et suit des procédures (SOP, playbooks). S'il y a un doute, il **escalade** avec ses notes.

Tier 2 : l'investigateur

Il reçoit un cas suspect et cherche l'histoire complète : il interroge plusieurs sources, corrèle, détermine l'étendue et la cause, et conclut si l'incident est réel. Il propose ou déclenche la réponse prévue par les procédures.

Tier 3 : l'expert

Il traite les incidents complexes : compromission avancée, malware, analyse forensique (étude des preuves numériques). Il pratique le **threat hunting** (recherche de menaces sans alerte préalable) et améliore les détections.

Autour des analystes

L'**Incident Responder** pilote la crise : confinement, éradication, récupération. Le **Detection Engineer** écrit et teste les règles de détection. Le **Security Engineer** déploie les capteurs et durcit les systèmes. Le **SOC Manager** priorise, suit les indicateurs et rend compte à la direction.

Quand escalader ?

Quand il y a un indice de compromission réelle, un actif critique concerné, ou un doute que l'analyste ne peut pas lever. Le dossier emporte l'alerte d'origine, le contexte récolté, les hypothèses et les actions déjà faites.

2.3 Les technologies d'un SOC

> Chaque technologie observe un endroit différent. Elles ne se remplacent pas : elles se complètent.

SIEM
Security Information and Event Management. Centralise les journaux de toute l'infrastructure, les normalise, les corrèle et lève des alertes. C'est le cœur du SOC.

EDR
Endpoint Detection and Response. Un agent installé sur le poste enregistre processus, fichiers modifiés et connexions, détecte les comportements anormaux et peut isoler la machine.

NDR
Network Detection and Response. Analyse le trafic réseau pour repérer des comportements suspects (fuite de données, balises régulières vers un serveur de contrôle).

XDR
Extended Detection and Response. Corrèle dans une seule plateforme les données des postes, du réseau, de la messagerie et du cloud.

SOAR
Security Orchestration, Automation and Response. Exécute automatiquement des playbooks (enrichissement, blocage, ticket) en reliant les outils entre eux.

TIP
Threat Intelligence Platform. Base d'indicateurs de compromission (IOC) : adresses, domaines, empreintes de fichiers connus comme malveillants.

Scanner de vulnérabilités
Parcourt les systèmes pour lister les failles connues (CVE) avant qu'un attaquant ne les exploite. Il agit en prévention.

Gestion de dossiers (case management)
Suit chaque incident de l'alerte à la clôture : preuves, chronologie, actions, intervenants.

IDS, EDR, NDR : trois angles de vue

L'IDS et le NDR regardent le **réseau**, l'EDR regarde la **machine**. Le SIEM reçoit les alertes et journaux des trois et les relie : c'est ce croisement qui sépare une vraie attaque d'une activité normale.

Partie 3

3 Les données et le SIEM

3.1 Quelles données un SOC exploite-t-il ?

Tout part d'une **trace**. Il en existe quatre familles, qu'il ne faut pas confondre.

Un **log** (journal) est une ligne écrite par un système : « tel utilisateur s'est connecté à telle heure ». Un **événement** est ce log une fois compris, avec ses champs (utilisateur, adresse, résultat). La **télémétrie** est un flux de données techniques détaillées envoyé en continu par un agent spécialisé : création de processus, fichiers modifiés, connexions ouvertes. Le **trafic réseau** est la copie des paquets observés ; on en tire des alertes (IDS) ou des **métadonnées** (qui a parlé à qui, quand, avec quel protocole).

Visibilité réseau contre **télémétrie endpoint** : le réseau est un observateur extérieur, indépendant de la machine même si elle est compromise, mais il ne lit pas le contenu chiffré. L'endpoint raconte ce que la machine fait, avec beaucoup de détails, mais il dépend de son agent.

```
Pare-feu : DROP SRC=192.168.163.136 DST=192.168.163.142 DPT=22
Serveur  : sshd[881]: Failed password for root from 192.168.163.136 port 51022
Windows  : EventID=4625 Account=admin LogonType=3 SourceIP=192.168.163.136
Web      : 192.168.163.136 "POST /login" 401 1532
```

Notre attaque vue par quatre sources : même acteur, quatre formats différents.

3.2 Du log à l'incident : le vocabulaire

> Six mots désignent six niveaux de valeur. Les confondre fait perdre du temps et de la clarté.

Terme	Définition	Dans notre exemple
Log	Trace brute écrite par un système.	Failed password for root from 192.168.163.136
Événement	Log compris : champs extraits par le parsing.	action = échec d'authentification, utilisateur = root, source = 192.168.163.136
Signal	Information potentiellement intéressante pour la sécurité.	200 échecs en une minute depuis une adresse inconnue
Alerte	Signal qui a satisfait une condition de détection (une règle).	Règle « brute force SSH » déclenchée
Incident	Menace confirmée ou assez probable pour exiger une réponse. Décision humaine.	Le compte root a été compromis
Dossier (case)	Regroupe preuves, chronologie et actions liées à l'incident.	Dossier 2026-041

Un log n'est pas une alerte, une alerte n'est pas un incident

La plupart des logs sont normaux. L'alerte naît d'une **condition** (une règle). L'incident naît d'un **jugement humain** après vérification.

La valeur augmente quand on relie les faits

10 échecs de connexion : un fait isolé, peut-être une faute de frappe. Avec une connexion réussie ensuite et une adresse inhabituelle : des faits reliés. Suivi d'un processus étrange sur le serveur : un comportement d'attaque.

3.3 Collecter et transmettre

La **collecte** consiste à récupérer les traces là où elles sont produites. Un **agent** est un petit programme installé sur la machine : il lit les journaux, ajoute du contexte (nom de la machine, système), garde les données en mémoire si le réseau tombe (*buffering*) et les envoie. C'est la méthode la plus complète, mais il faut le déployer et le maintenir. Sans agent, la machine envoie elle-même ses journaux par **syslog** (protocole standard des équipements réseau et de Linux), ou le SIEM interroge une **API** (obligatoire pour le cloud). Sous Windows, **WEF** (Windows Event Forwarding) transfère les événements vers un collecteur central.

La transmission peut être en **push** (la source envoie : réactif) ou en **pull** (le SIEM va chercher : maîtrisé, avec un délai). Un **relais** peut regrouper les logs d'un site avant de les envoyer au centre.

Le transport : UDP, TCP, TLS

UDP est rapide mais sans accusé de réception : un message perdu passe inaperçu. **TCP** garantit la livraison. **TLS** chiffre et authentifie le transport, empêchant l'écoute ou la falsification en route. Les logs importants devraient circuler en TCP avec TLS.

Les risques

Perte et latence : un log arrivé trop tard ou jamais est une attaque non vue. Volume : plus on collecte, plus on stocke. **Intégrité** : un attaquant qui compromet une machine peut effacer ses journaux locaux ; il faut donc les envoyer ailleurs, vite.

Exemple

Le serveur 192.168.163.142 écrit « Failed password » dans son journal d'authentification. L'agent le lit en quelques secondes et l'envoie en connexion chiffrée au serveur central. Même si l'attaquant effaçait ensuite le fichier, la trace est déjà ailleurs.

3.4 Le SIEM : rôle et composants

Un **SIEM** (Security Information and Event Management) réunit deux métiers : le **SIM** (gérer l'information : stocker et analyser l'historique) et le **SEM** (gérer les événements : surveiller en temps réel). Il répond à un problème simple : personne ne peut relier à la main des millions de lignes issues de sources différentes. Il travaille jour et nuit, mais ne juge pas : des humains écrivent ses règles et traitent ses alertes.

Quel que soit le produit, on retrouve les mêmes composants. Les **collecteurs et agents** récupèrent les données. Un **moteur d'ingestion** les reçoit, les décode et les normalise. Un **stockage indexé** permet de les rechercher en quelques secondes parmi des milliards de lignes. Un **moteur de corrélation et de règles** détecte les situations suspectes. Une **interface** (tableaux de bord, recherche) sert à l'investigation. Des **connecteurs** envoient les alertes vers les tickets, les e-mails ou l'automatisation.

Des exemples concrets : **Wazuh** et **Security Onion** (open source, détaillés en partie 4), ou, côté commercial, Splunk, IBM QRadar et Microsoft Sentinel.

Ce que le SIEM ne fait pas

Il ne bloque rien par lui-même et ne remplace pas la prévention. Et il suit la règle « garbage in, garbage out » : si la donnée n'arrive pas, ou n'est pas bien lue, la meilleure règle reste muette.

3.5 Dans le SIEM : le pipeline, étape par étape

1. Ingestion. La donnée arrive et est horodatée. Si le SIEM est saturé, des événements sont perdus.

2. Parsing. Un *décodeur* découpe la ligne brute en champs (heure, adresse source, utilisateur, action). Sans parsing, une règle ne peut pas lire les valeurs.

3. Normalisation. On donne à chaque champ un nom commun, quelle que soit la source. Un échec SSH sous Linux et un événement 4625 sous Windows deviennent la même action « échec d'authentification » : une seule règle couvre les deux.

4. Indexation. Les événements sont stockés de façon à être retrouvés vite.

5. Enrichissement. On ajoute du contexte : pays de l'adresse, réputation, criticité de la machine, identité de l'utilisateur. Sans lui, impossible de savoir si l'actif concerné est critique.

6. Corrélation. On relie plusieurs événements : même source, même compte, fenêtre de temps courte, plusieurs sources.

7. Détection. Une règle, un comportement ou une anomalie identifie un motif suspect.

8. Sévérité. On attribue un niveau de gravité.

9. Alerte. Un signal exploitable est créé et transmis aux analystes.

```
Brut        : May 4 10:02:11 srv1 sshd[881]: Failed password for root from 192.168.163.136
port 51022
Normalisé   : heure=10:02:11 · hôte=srv1 · action=échec_auth · user=root ·
src_ip=192.168.163.136 · port=51022
Enrichi     : src_ip = adresse externe, mal réputée · hôte = serveur critique
```

Détection ratée : remonter la chaîne

Le problème est rarement la règle seule : souvent la donnée n'est jamais arrivée, ou n'a pas été bien lue.

3.6 Comment le SIEM détecte

La **règle (ou signature)** cherche un motif connu : « plus de 10 échecs d'authentification en une minute depuis une même source ». Elle est précise, mais aveugle à ce qu'on n'a pas prévu. L'**IOC** (Indicator of Compromise) est une trace connue d'une attaque : adresse, domaine, empreinte de fichier. La détection par IOC est utile tant que la liste est à jour.

La **détection comportementale** cherche des séquences d'actions typiques d'une attaque, même avec des outils inconnus. La **détection d'anomalies** compare l'activité à une référence normale (*baseline*) et signale les écarts ; elle trouve l'inattendu mais génère du bruit.

MITRE ATT&CK : un langage commun

MITRE ATT&CK est une base de connaissances qui décrit comment les attaquants opèrent. Une **tactique** est l'objectif (pourquoi) : accès initial, exécution, persistance, découverte, mouvement latéral, exfiltration. Une **technique** est la méthode (comment), avec un identifiant : T1110 pour la force brute, T1046 pour la découverte de services réseau. Une **sous-technique** précise le détail. Une **procédure** est la façon exacte dont un attaquant donné applique la technique. Rattacher chaque règle à une technique permet de savoir ce qu'on couvre et ce qu'on ne couvre pas.

Corrélation sur notre exemple

Cinq échecs, puis une connexion réussie, depuis une adresse jamais vue, puis un processus inhabituel sur le serveur. Chaque fait seul est faible ; ensemble, ils forment une alerte à forte confiance, rattachée à T1110.

3.7 Les alertes : vrai, faux, positif, négatif

> Une détection peut avoir raison ou se tromper, dans les deux sens. Quatre cas existent.

	Il y a une attaque	Il n'y a pas d'attaque
Une alerte est levée	**Vrai positif.** Bonne détection. Ex. l'alerte de force brute sur l'attaque réelle.	**Faux positif.** Fausse alerte. Ex. un utilisateur qui se trompe dix fois de mot de passe.
Aucune alerte	**Faux négatif.** Attaque non vue : le cas le plus dangereux.	**Vrai négatif.** Activité normale, silence normal.

Trop de faux positifs conduisent à la **fatigue d'alerte** : les analystes finissent par ignorer les alertes, y compris les vraies. Trop peu de sensibilité crée des faux négatifs. Le **réglage** (*tuning*) cherche l'équilibre : un bon SOC ne vise pas le plus d'alertes, mais des alertes pertinentes, contextualisées et exploitables.

Il faut distinguer la **sévérité** (gravité technique de ce qui est détecté) de la **priorité** (urgence de traitement, qui tient compte du contexte). Le **risque** combine la probabilité et l'impact : une alerte identique n'a pas le même poids sur un serveur critique que sur une machine de test.

La priorité dépend de la criticité de l'actif, de la confiance de la détection, du compte concerné, de l'impact potentiel, de l'exposition à Internet et du renseignement disponible.

Notre exemple

Dix échecs depuis l'adresse d'un collègue ressemblent à un faux positif. Deux cents échecs depuis une adresse inconnue, suivis d'un succès sur le compte root d'un serveur critique, forment un vrai positif de haute priorité.

Partie 4

Les outils open source

> Pour chaque outil : où il se place, ce qu'il reçoit, ce qu'il produit, et qui s'en sert.

Wazuh : SIEM et XDR

Position : plateforme centrale. **Entrée :** journaux et événements envoyés par des agents installés sur les machines, ainsi que des journaux reçus par syslog. **Traitement :** le *serveur Wazuh* décode les événements et applique des règles de détection ; l'*indexeur* les stocke pour la recherche ; le *tableau de bord* les affiche. L'agent surveille aussi l'**intégrité des fichiers** (FIM : détecte toute modification d'un fichier sensible), évalue la configuration de sécurité et détecte les vulnérabilités du système. Il peut exécuter une **réponse active** : par exemple bloquer une adresse localement. **Sortie :** alertes classées par niveau, avec leur contexte. **Utilisateurs :** Tier 1 et Tier 2.

Security Onion : surveillance réseau en distribution complète

C'est une distribution Linux prête à l'emploi, orientée **surveillance de la sécurité du réseau**. Elle rassemble des capteurs (Suricata pour les alertes, Zeek pour les métadonnées), la capture complète des paquets pour pouvoir y revenir, une interface de recherche et de gestion de cas. Elle est plus gourmande en ressources et moins modulaire, mais elle fournit une chaîne complète sans assemblage.

Comment choisir ?

Wazuh excelle sur la visibilité des **machines** (agents, fichiers, vulnérabilités). Security Onion excelle sur la visibilité du **réseau**. Les deux sont complémentaires.

4.2 Les sondes réseau : Suricata, Zeek, Wireshark

Suricata

Position : sur l'interface de capture qui reçoit la copie du trafic. **Entrée :** les paquets. **Traitement :** il les compare à des règles (signatures) et reconnaît aussi les protocoles (HTTP, DNS, TLS). Il fonctionne en mode IDS (alertes seules) ou IPS (en ligne, blocage). **Sortie :** des alertes réseau, écrites au format JSON (fichier `eve.json`) que Wazuh peut lire. Sa limite : il consomme de la mémoire et ses règles doivent être réglées pour limiter les fausses alertes.

Zeek

Zeek ne cherche pas des signatures. Il **décrit** ce qui se passe : pour chaque connexion, chaque requête DNS, chaque échange HTTP ou TLS, il écrit une ligne de métadonnées. **Sortie :** des journaux riches qui permettent de remonter le fil d'une investigation et de chasser des menaces. Il est moins immédiat à prendre en main.

Wireshark

Wireshark n'est pas un capteur permanent mais un outil d'**analyse manuelle** : on ouvre une capture (un fichier PCAP) et on lit les paquets un à un. L'analyste l'utilise à la fin d'une investigation pour vérifier un détail précis : que contenait réellement cet échange ?

Complémentaires

Suricata **alerte** sur ce qu'il connaît, Zeek **garde la mémoire** de tout ce qui s'est passé, Wireshark **explique** un échange précis. Le SIEM reçoit leurs résultats : jamais le flux de paquets bruts.

4.3 Gestion d'incident, enrichissement, automatisation

TheHive : les dossiers

Plateforme collaborative de **gestion d'incidents**. Chaque incident y devient un dossier (case) avec ses tâches, ses preuves et ses **observables** (une adresse, un domaine, un fichier à analyser). Plusieurs analystes y travaillent ensemble et on garde l'historique. Vérifiez la licence actuelle de l'édition utilisée.

Cortex : l'analyse automatisée

Cortex prend un observable et le soumet à des **analyseurs** (VirusTotal pour la réputation d'un fichier ou d'une adresse, par exemple). Il renvoie un rapport qui s'ajoute au dossier. Ses *responders* peuvent aussi déclencher des actions.

MISP : le renseignement partagé

Plateforme de partage d'**IOC** et de renseignement sur les menaces, au sein d'une communauté. Elle sert à comparer ce qu'on observe avec ce que d'autres ont déjà vu.

Shuffle : le SOAR

Il relie les outils par des **workflows** visuels : à la réception d'une alerte, on enchaîne des actions (interroger Cortex, créer le dossier TheHive, notifier, bloquer).

OpenVAS (Greenbone) : les vulnérabilités

Scanner qui parcourt les machines et liste leurs failles connues. Il donne une vue **préventive** : on corrige avant d'être attaqué, et on sait quelles machines sont les plus exposées.

4.4 Qui envoie quoi à qui ?

> Ce schéma résume tout le chemin de la donnée, des sources jusqu'à l'analyste.

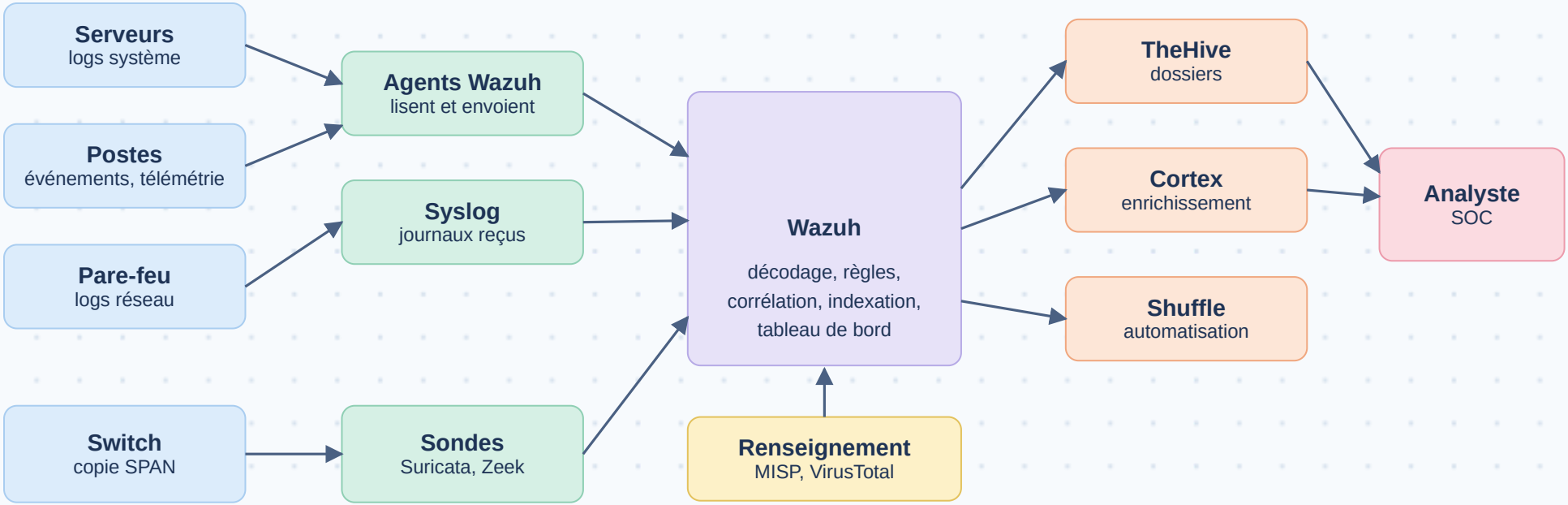


Figure 5 : *Les sources, la collecte, la plateforme, les outils de réponse, puis l'analyste.*

Cinq flux différents

Télémétrie endpoint : machine, agent, Wazuh. **Logs** : fichiers ou syslog vers Wazuh. **Trafic réseau** : switch, SPAN, sonde, puis résultats vers Wazuh. **Renseignement** : IOC qui enrichissent les événements. **Automatisation** : alerte, workflow, action, retour vers l'analyste.

Partie 5

5 L'humain et l'automatisation

5.1 Chronologie d'une attaque, de bout en bout

T0 L'attaquant (192.168.163.136) scanne le port 22 du serveur 192.168.163.142 (reconnaissance).

T+1 Il lance des essais de mots de passe SSH (force brute, T1110).

T+2 Le serveur écrit un « Failed password » par tentative.

T+3 L'agent lit le journal et envoie les événements.

T+4 Le serveur Wazuh reçoit, décode et normalise.

T+5 La sonde Suricata, de son côté, alerte sur les connexions répétées vues via le SPAN.

T+6 Le moteur de règles compte les échecs sur la fenêtre de temps.

T+7 Une connexion réussie depuis la même adresse élève la confiance.

T+8 Une alerte de haute sévérité est créée.

T+9 Tier 1 la voit dans la file et la vérifie.

T+10 Il enrichit : adresse inconnue, mal réputée, serveur critique. Il escalade.

T+11 Tier 2 corrèle : processus lancé après la connexion, fichiers modifiés.

T+12 L'incident est confirmé : compte compromis.

T+13 Containment : adresse bloquée, compte désactivé.

T+14 Éradication : mot de passe changé, accès fermé, persistance supprimée.

T+15 Recovery : service remis en route, surveillance renforcée.

T+16 Retour d'expérience : pourquoi pas plus tôt ?

T+17 Nouvelle règle de détection, ajustements.

5.2 Le travail de l'analyste

Le triage (Tier 1). L'analyste ouvre l'alerte dans le tableau de bord. Il lit ce qui est détecté, puis vérifie les **faits** : quelle adresse, quel compte, quelle machine ? Est-ce connu ? Le compte a-t-il l'habitude de se connecter ainsi ? Il enrichit avec la réputation de l'adresse et la criticité du serveur. Il qualifie : *bénin* (légitime), *suspect* ou *malveillant*. Si c'est suspect, il escalade avec ses notes.

L'investigation (Tier 2). Une alerte donne un signal ; l'investigation cherche l'histoire complète. L'analyste se pose les mêmes questions : **qui** (quel compte), **quoi** (quel processus, quel fichier), **quand** (avant et après l'alerte), **d'où** (adresse source), **vers où** (destination, port, protocole), **quelle machine**, et quels **autres événements** y sont liés.

Il utilise les outils pour répondre : le tableau de bord pour rechercher tous les événements de cette adresse sur la période, les journaux Zeek pour voir les connexions réseau, Suricata pour les alertes du même instant, TheHive pour consigner chaque constat. Si un point reste obscur, il ouvre la capture de paquets dans **Wireshark** et lit l'échange. Il reconstitue ainsi la chronologie : accès initial, actions, étendue, éventuelle persistance.

Les preuves

Pour qu'une conclusion soit fiable, les horloges des machines doivent être synchronisées (NTP), les journaux protégés, les fichiers identifiés par leur empreinte (hash), et chaque action de l'analyste notée.

5.3 La réponse à incident

> Une fois l'incident confirmé, on suit un cycle structuré (inspiré du guide NIST SP 800-61).

Préparation

Avant l'incident : procédures, outils, contacts, entraînement. Sans elle, on improvise sous pression.

Identification

On confirme qu'il s'agit d'un incident, on mesure son étendue et sa gravité.

Containment (confinement)

Limiter les dégâts sans détruire les preuves : bloquer l'adresse, isoler la machine, désactiver le compte.

Éradication

Supprimer la cause : réinitialiser le mot de passe, fermer l'accès, retirer un logiciel malveillant, corriger la faille.

Recovery (récupération)

Remettre en service, vérifier que tout est revenu à la normale, surveiller de près le retour.

Lessons learned

Documenter, comprendre ce qui a marché ou non, ajuster règles et procédures.

Ne pas confondre

Contenir arrête la propagation ; **éradiquer** supprime la cause ; **récupérer** rétablit le service. Restaurer trop vite sans éradiquer relance l'incident ; éradiquer avant de contenir laisse l'attaque se propager.

Notre exemple

Contenir : bloquer 192.168.163.136 sur le pare-feu et désactiver le compte root. Éradiquer : changer le mot de passe, interdire la connexion root par SSH, supprimer ce que l'attaquant a laissé. Récupérer : remettre le serveur en service sous surveillance renforcée.

5.4 Automatisation, SOAR et intelligence artificielle

Un SOC reçoit énormément d'alertes. Beaucoup de tâches sont répétitives : chercher la réputation d'une adresse, ouvrir un ticket, notifier. Le **SOAR** les exécute automatiquement. Un **playbook** est une procédure écrite sous forme d'étapes : « si telle alerte, alors faire ceci, puis cela ». Il transforme une réaction improvisée en procédure reproductible.

Exemple de playbook sur notre attaque : l'alerte arrive ; le SOAR récupère l'adresse ; il interroge la réputation (Cortex, VirusTotal, MISP) ; si la réputation est mauvaise *et* que le comportement est confirmé, il bloque l'adresse sur le pare-feu ; il crée le dossier TheHive ; il notifie l'analyste ; il journalise ses actions. L'analyste vérifie ensuite le résultat.

Qui décide ?

Automatique : détection, création d'alerte, enrichissement, blocage d'une adresse à faible risque. **Hybride** : le système prépare et l'analyste valide (investigation assistée, proposition d'isolement). **Humain** : qualification finale, isolement d'un serveur critique, toute décision à fort impact métier. L'automatisation n'est pas « laisser la machine tout décider » : le playbook fixe ce qui est permis sans validation.

L'intelligence artificielle

L'IA assiste : elle résume des incidents, classe et priorise, repère des anomalies, aide à écrire des requêtes de recherche, propose des hypothèses. Ses limites : faux positifs et faux négatifs, manque de contexte, erreurs inventées (hallucinations). Le principe est le **human-in-the-loop** : la machine détecte et propose, l'humain vérifie, puis décide pour les actions critiques.

5.5 L'amélioration continue : la défense proactive

Un incident clos n'est pas la fin. C'est la meilleure source d'amélioration. Le **retour d'expérience** pose des questions précises : pourquoi la détection a-t-elle fonctionné ? Pourquoi pas plus tôt ? Quelle visibilité manquait ? Quelle règle faut-il corriger ? Quel playbook faut-il modifier ?

Le **Detection Engineering** transforme les réponses en règles : créer, tester et affiner les détections, rattachées à une technique MITRE ATT&CK, et réduire les faux positifs. Le **threat hunting** cherche, sans attendre une alerte, des signes de compromission à partir d'une hypothèse : « un attaquant qui a réussi une force brute aurait créé un compte ou un service ; vérifions ». La **gestion des vulnérabilités** corrige les failles avant qu'elles soient exploitées. Le **purple teaming** fait travailler ensemble ceux qui attaquent (red team) et ceux qui défendent (blue team) pour tester que les détections fonctionnent vraiment.

Dans notre exemple : on ajoute une règle qui signale toute connexion root réussie après une rafale d'échecs, on active le blocage automatique, on durcit le SSH (clés, pas de root), et on cherche si d'autres serveurs ont reçu la même rafale.

Une boucle, pas une ligne droite

Détection, analyse, amélioration, nouvelle détection, meilleure protection. À chaque tour, le SOC détecte plus vite et plus juste que le précédent.

Laboratoire contre SOC réel

Un laboratoire de quelques machines reproduit la chaîne à petite échelle : un opérateur joue tous les niveaux, les menaces sont simulées. Un SOC réel ajoute un volume énorme, une équipe en niveaux, la production et l'automatisation. Les principes sont identiques.

Partie 6

Mise en pratique : notre laboratoire

> Un laboratoire personnel reproduit à petite échelle la chaine vue en théorie : un SIEM Wazuh, une machine victime, un attaquant Kali, et le même scénario de force brute SSH (T1110) que notre exemple fil rouge.

Avertissement sur l'échelle

Un opérateur joue ici tous les rôles (Tier 1, Tier 2, attaquant) dans un environnement isolé, avec un trafic simulé. Les mécanismes observés sont réels ; le volume et l'organisation en équipe d'un SOC réel ne le sont pas (cf. 5.5).

6.1 Le scénario

> Un manager Wazuh est installé, une machine est mise sous surveillance, puis attaquée par force brute sur SSH — et l'attaque est détectée en quelques secondes. Trois scripts d'automatisation ont servi : un pour le manager, un pour l'agent local, un pour l'agent de la victime. Voici le déroulé réel, capture après capture.

UB1 — serveur & manager

Héberge le manager Wazuh, l'indexer et le dashboard. Reçoit aussi un agent local.

serv — machine surveillée

Agent Wazuh installé dessus : c'est la cible de l'attaque par force brute SSH.

Kali — machine attaquante

Hors ligne, horloge non synchronisée : elle lance l'attaque à l'aide de Hydra.

4 oct. Installation du manager Wazuh sur UB1

5 oct. 18h Déploiement de l'agent sur la machine cible

5 oct. 18h47 Attaque par force brute lancée depuis Kali

5 oct. 19h54 Alerte détectée et qualifiée dans le dashboard

6.2 Automatisation : installer le manager

UB1 **Script 1 · 01_manager_ub1.sh · 192.168.163.135**

> Un seul script, lancé sur UB1, installe tout le SIEM puis relie la sonde Suricata. Il s'arrête à la première erreur et peut être relancé : sa première étape nettoie une éventuelle installation précédente.

Étape	Ce que fait le script
0. Nettoyage	Si des paquets wazuh ou filebeat existent déjà : purge complète et suppression des dossiers résiduels.
1. Système	Mise à jour, puis installation de curl, wget, tcpdump, ethtool, net-tools et suricata.
2. Wazuh	Télécharge `wazuh-install.sh` (version 4.14) et lance l'installation tout-en-un : manager, indexer, dashboard.
3. Mots de passe	Extrait `wazuh-passwords.txt` de l'archive générée, en lecture réservée à l'utilisateur.
4. Contrôle	Vérifie que les trois services Wazuh sont `active`.
5. Capture	Interface `ens37` en mode promiscuous, puis test tcpdump de 10 secondes.
6. Suricata	HOME_NET = 192.168.163.0/24, interface de capture, règles à jour, test de configuration, activation.
7. Intégration	Ajoute `eve.json` de Suricata à la configuration du manager, puis redémarre le manager.

```
curl -sO https://packages.wazuh.com/4.14/wazuh-install.sh
sudo bash ./wazuh-install.sh -a

sudo systemctl is-active wazuh-manager wazuh-indexer wazuh-dashboard

sudo ip link set ens37 promisc on
sudo suricata-update
sudo suricata -T -c /etc/suricata/suricata.yaml
```

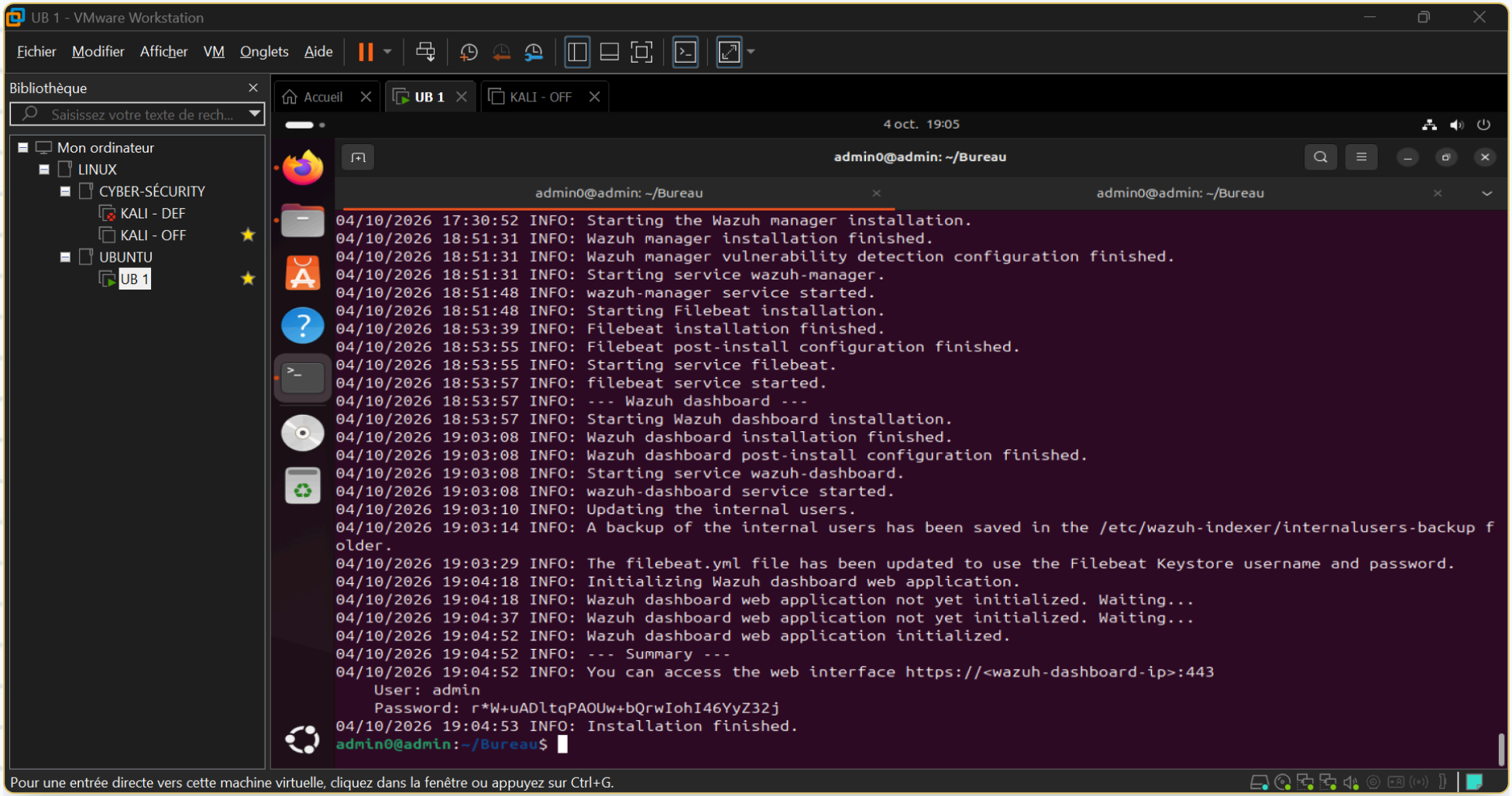
À retenir

Les étapes 2 à 4 produisent ce que montrent les captures qui suivent : résumé d'installation, identifiants admin, trois services actifs.

6.3 Installation du manager : le journal du script

UB1 Script 1 · étape 2

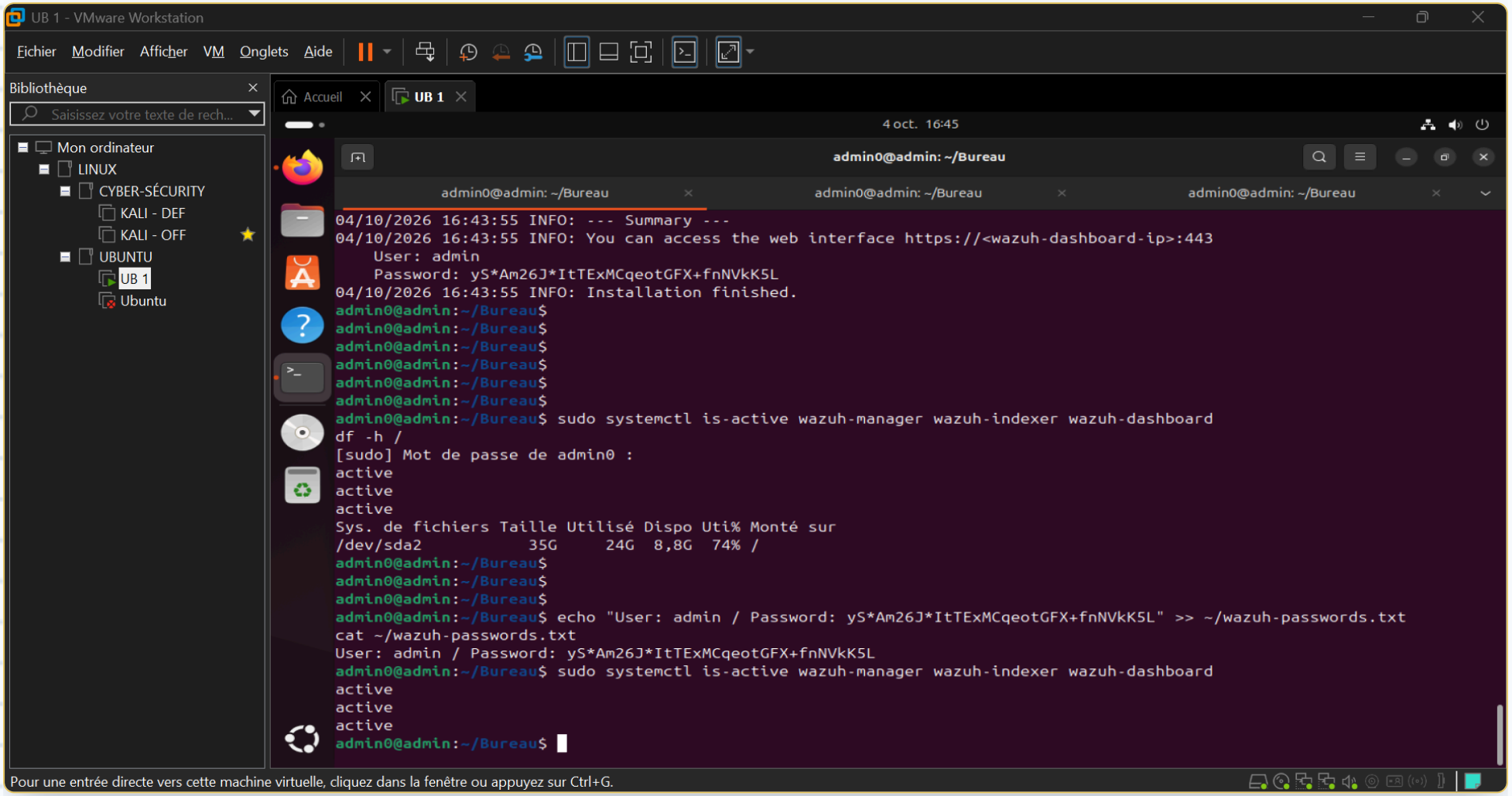
> Le script lance l'installation tout-en-un : le journal montre l'installation du manager, de Filebeat puis du dashboard, et se termine par le résumé avec le compte admin.



6.4 Installation terminée : les trois services sont actifs

UB1 4 oct. · 16:45

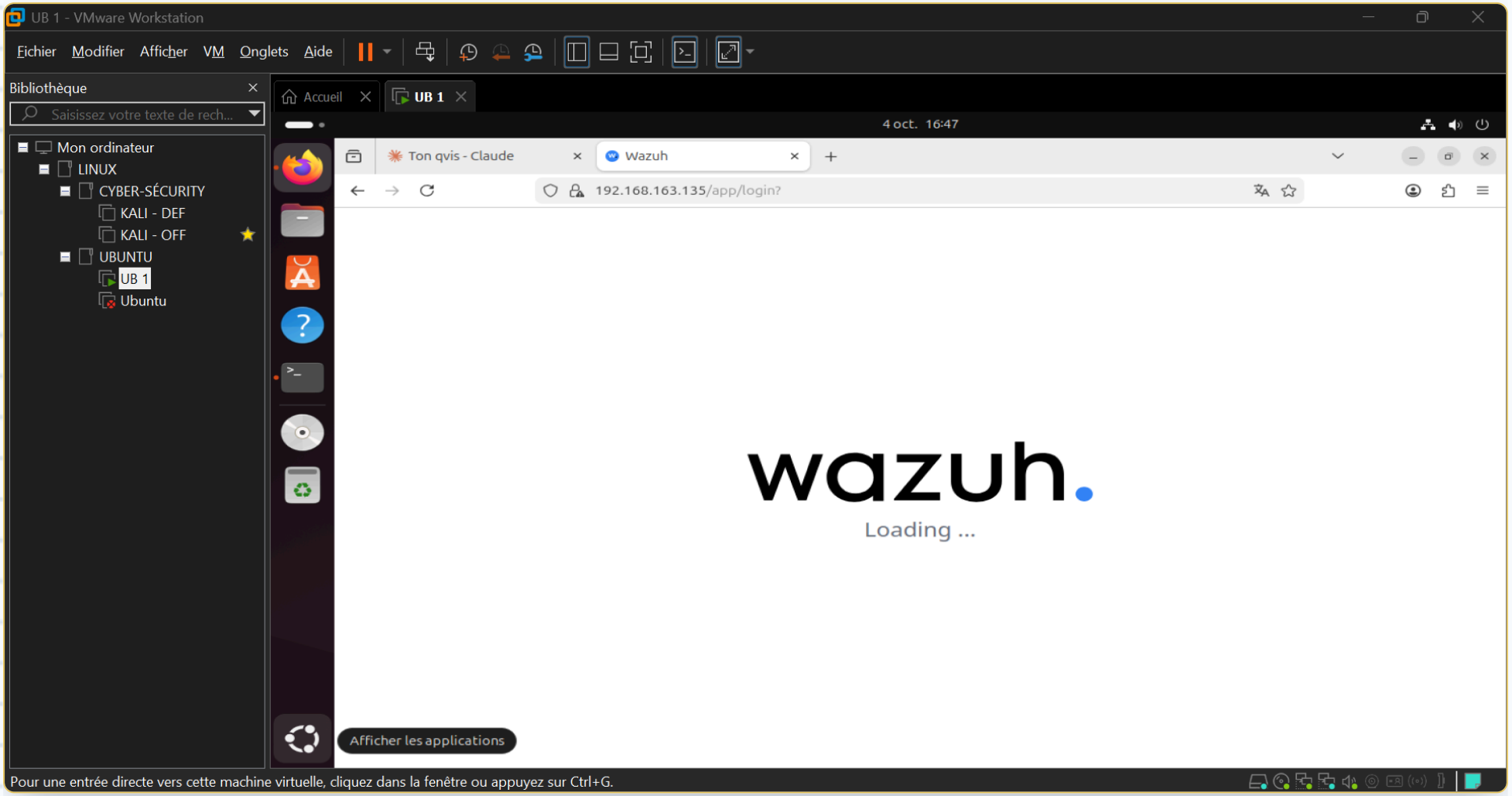
> Résultat des étapes 2 à 4 du script 1 : résumé d'installation, identifiants admin générés, puis les trois services confirmés `active`.



6.5 Premier accès au dashboard

UB1 4 oct. · 16:47

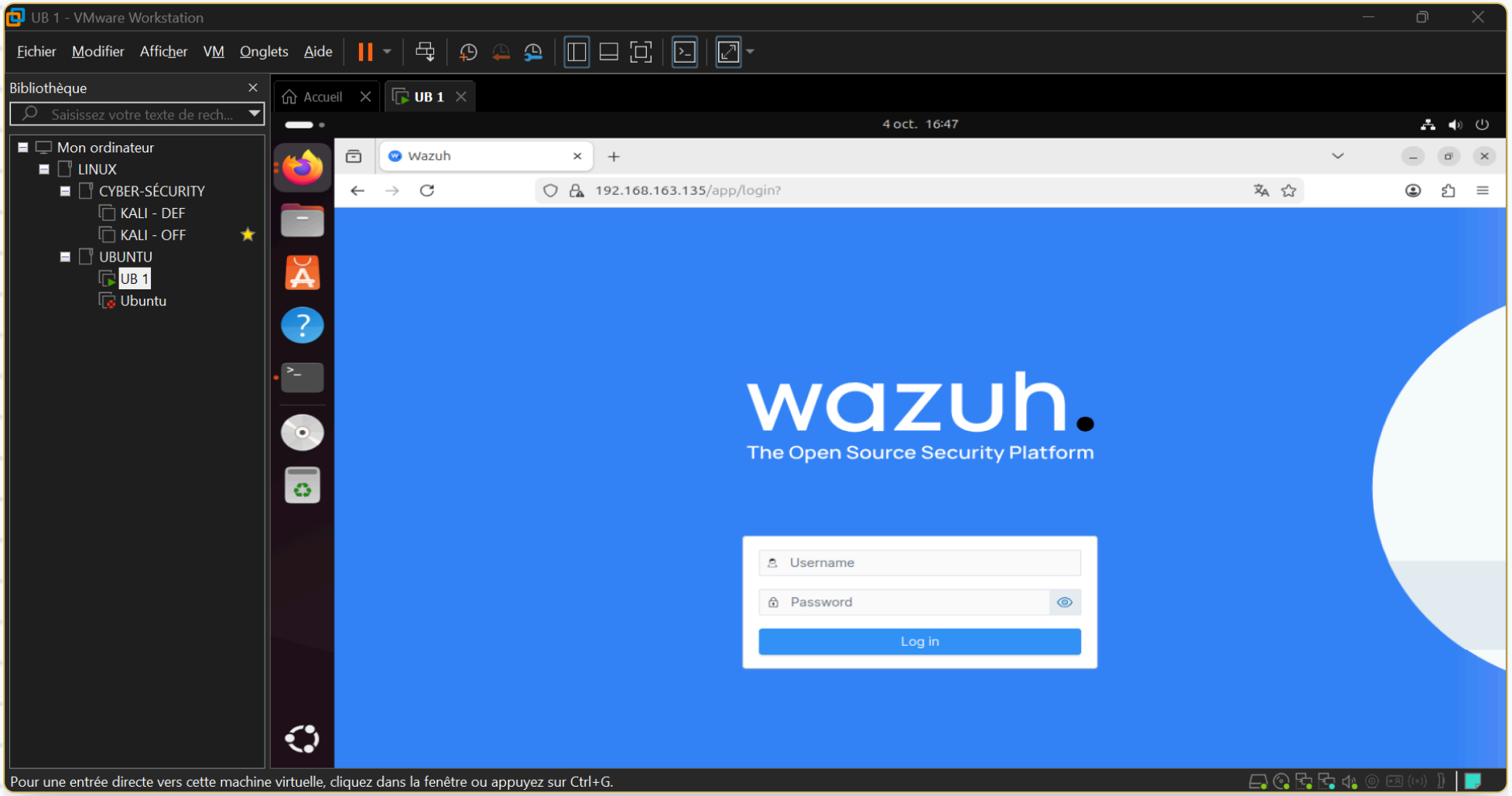
> Ouverture de l'interface Wazuh dans Firefox, sur UB1 lui-même.



6.6 La page de connexion

UB1 4 oct. · 16:47

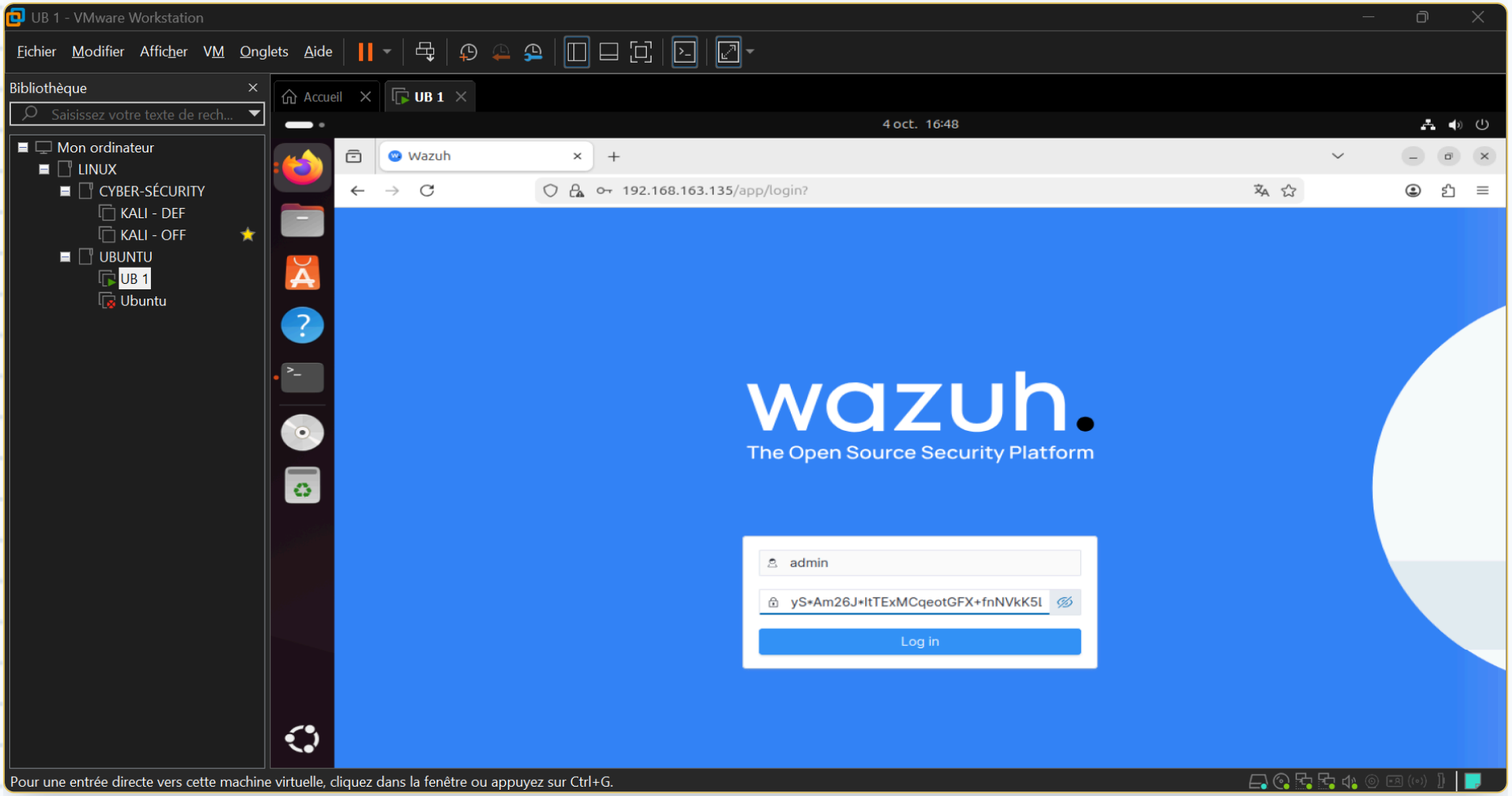
> Formulaire de login Wazuh, prêt à recevoir les identifiants générés.



6.7 Authentification

UB1 **4 oct. · 16:48**

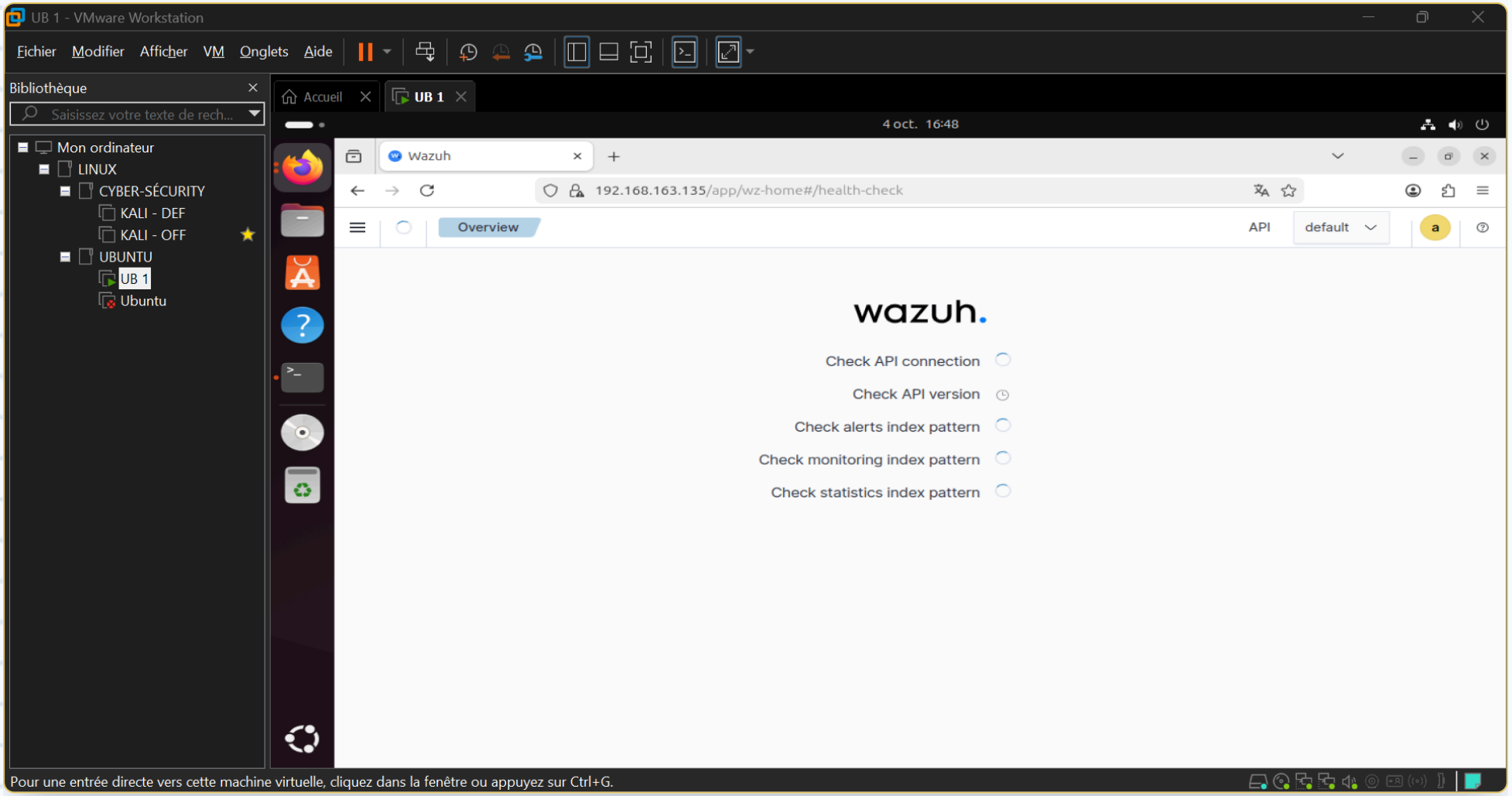
> Connexion avec le compte `admin` et le mot de passe généré à l'installation.



6.8 Vérifications internes

UB1 **4 oct. · 16:48**

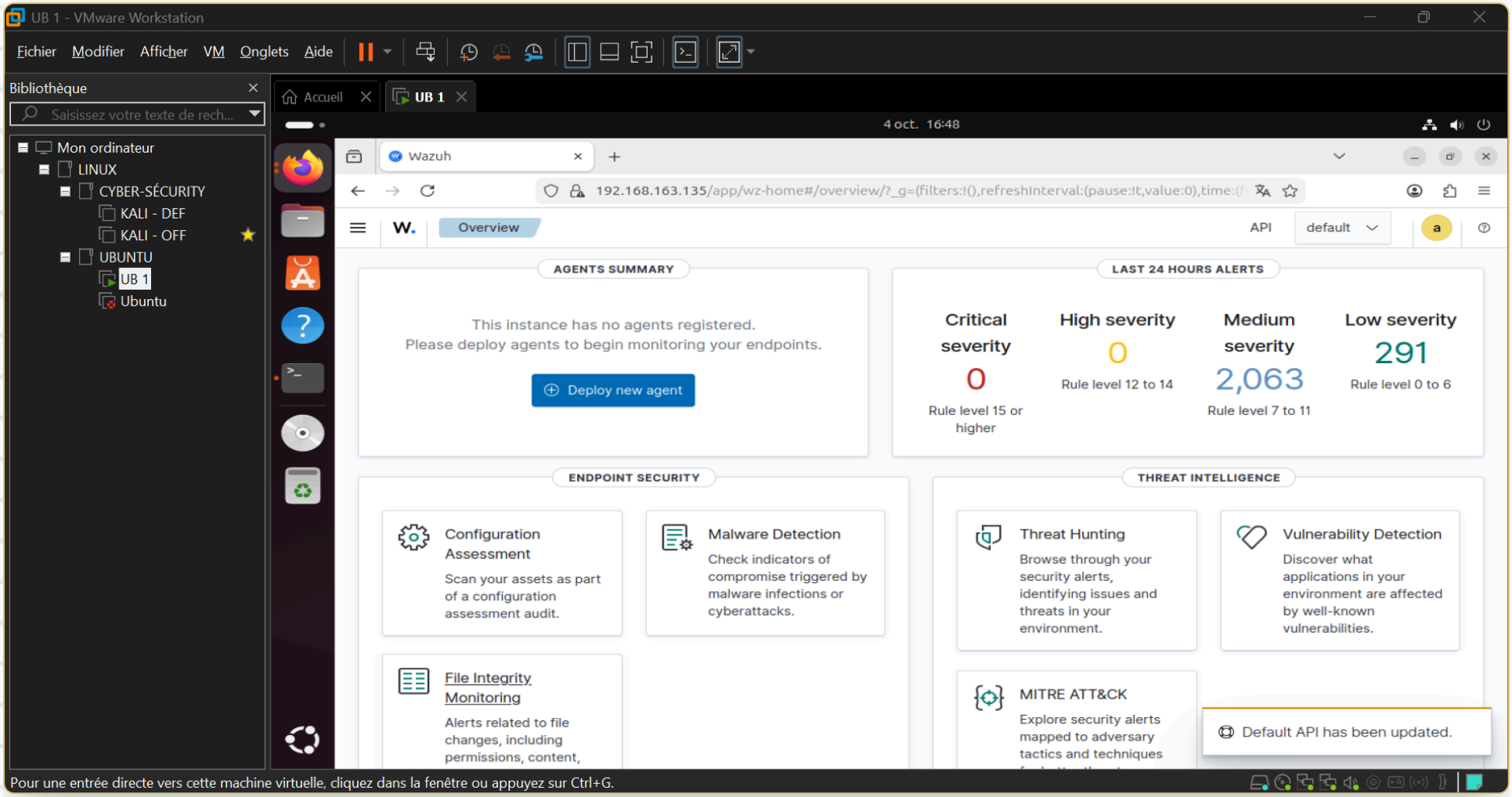
> Wazuh exécute son health-check : connexion à l'API, index patterns des alertes, du monitoring et des statistiques.



6.9 Un dashboard encore vide

UB1 4 oct. · 16:48

> Aucun agent enregistré pour l'instant — seules les alertes générées par le manager lui-même apparaissent.



6.10 Automatisation : l'agent local sur UB1

UB1 **Script 2 · 02b_agent_local_ub1.sh · UB1 elle-même**

> Solution provisoire : faute de place pour une machine victime séparée (KALI-DEF supprimée), l'agent est installé sur UB1 elle-même et pointe vers 127.0.0.1. Le script prépare aussi SSH (cible de la force brute) et Apache.

Étape	Ce que fait le script
0. Nettoyage	Arrête et purge un agent Wazuh déjà présent.
1. Paquets	Mise à jour des listes de paquets.
2. Dépôt	Ajoute le dépôt Wazuh et sa clé GPG, s'ils ne sont pas déjà là.
3. Agent	Installe `wazuh-agent` avec le manager 127.0.0.1, l'active au démarrage et vérifie qu'il est `active`.
4. SSH	Installe `openssh-server` : le service attaqué par force brute.
5. Apache	Installe `apache2`.

```
sudo WAZUH_MANAGER="127.0.0.1" apt
install -y wazuh-agent
sudo systemctl enable --now wazuh-
agent
sudo systemctl is-active wazuh-agent

sudo apt install -y openssh-server
sudo apt install -y apache2

sudo /var/ossec/bin/agent_control -l
```

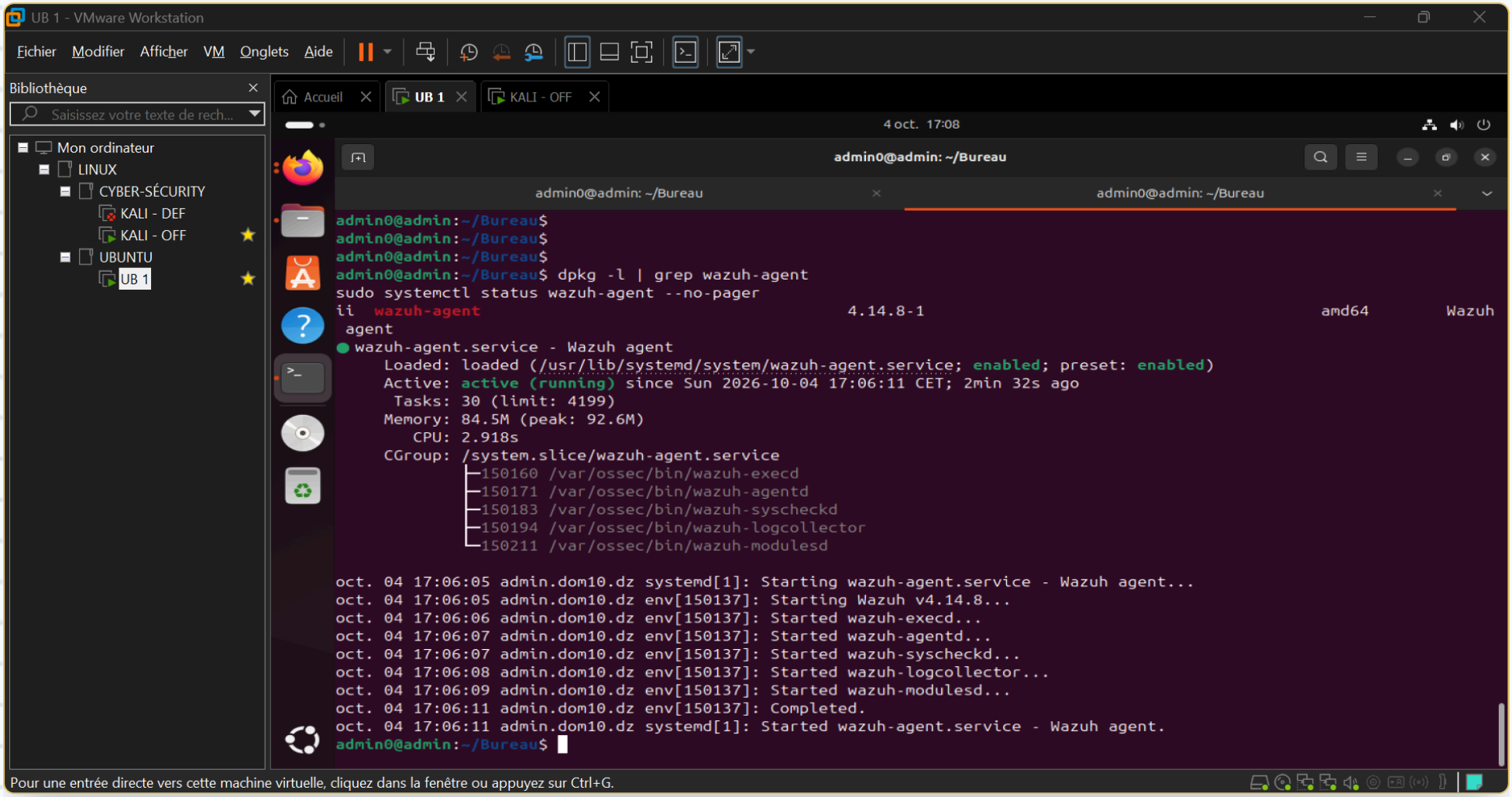
À retenir

La dernière commande liste les agents connus du manager : c'est la vérification proposée à la fin du script.

6.11 L'agent local tourne

UB1 4 oct. · 17:08

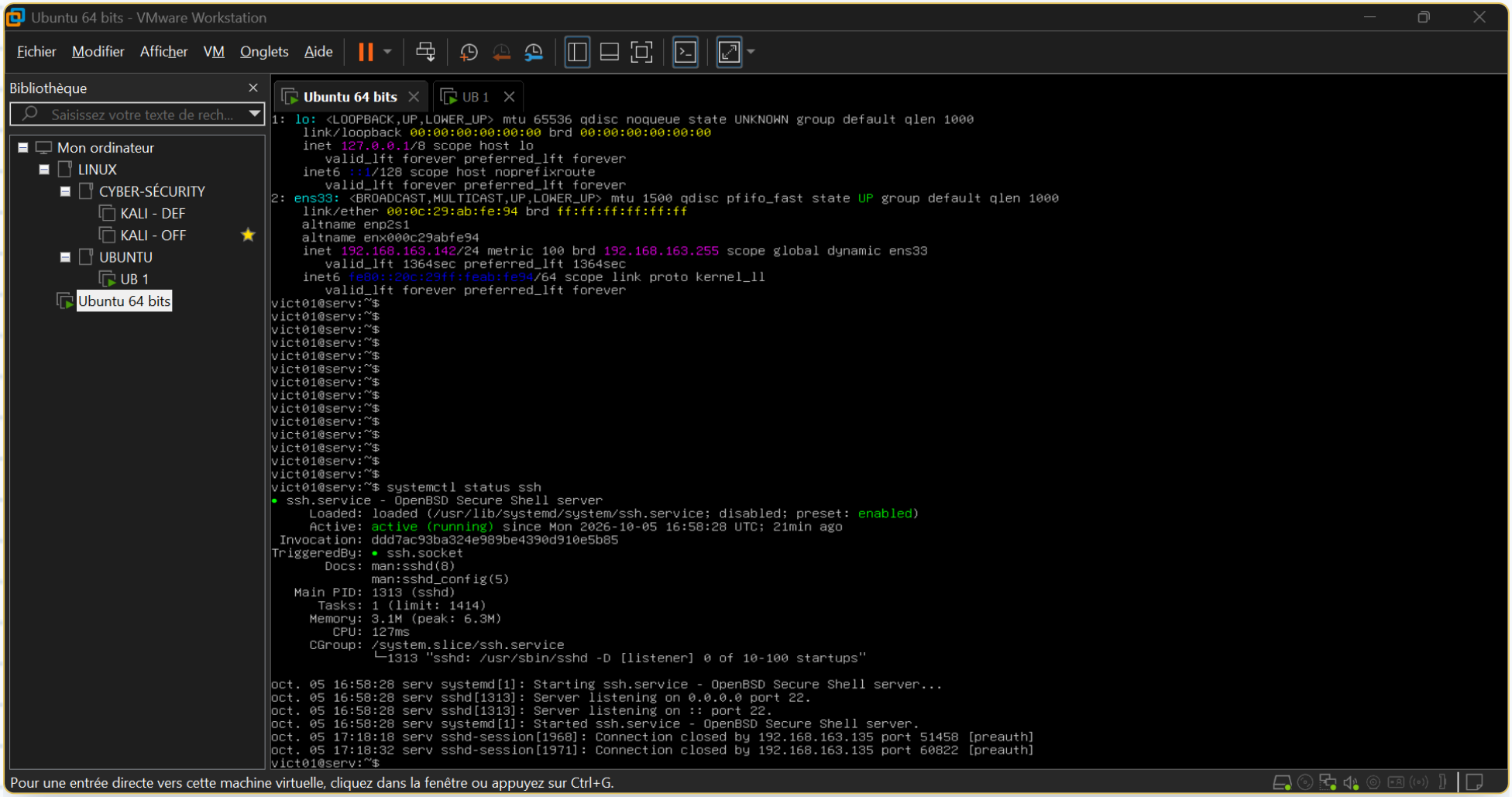
> Résultat du script 2 : l'agent Wazuh installé sur UB1 fonctionne : actif depuis 17h06.



6.12 La cible est prête

SERV 5 oct. · 18:20

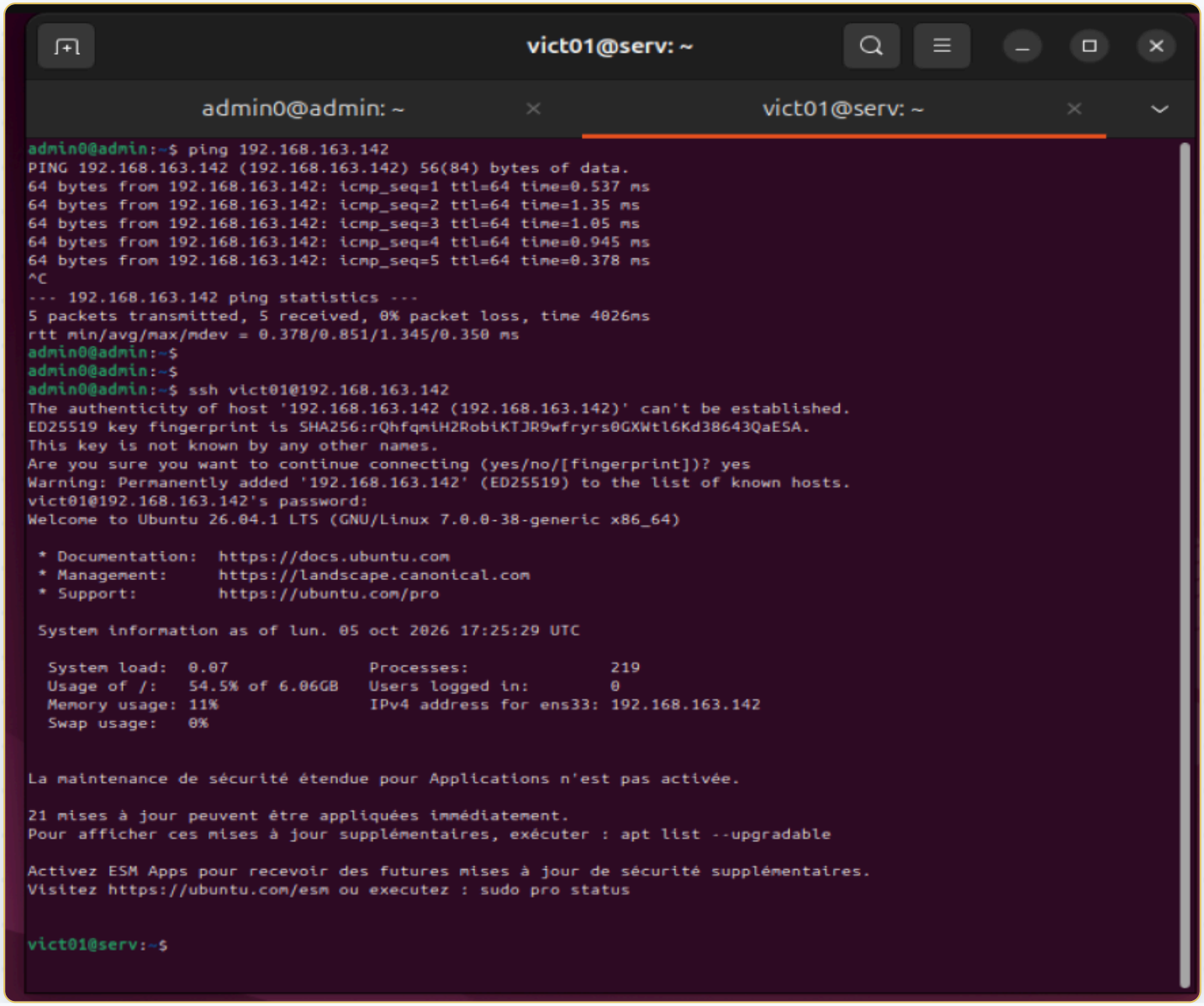
> Sur la machine **serv** : adresse 192.168.163.142 et service SSH actifs.



6.13 Connexion vers la cible

UB1 5 oct. · 18:27

> Depuis UB1 : un `ping` confirme la route, puis une première connexion SSH vers serv.



6.14 Automatisation : l'agent de la machine victime

SERV **Script 3 · 02c_agent_vict01.sh · 192.168.163.142**

> Lancé sur la machine victime (utilisateur vict01, adresse 192.168.163.142), le script installe l'agent et le pointe vers le manager UB1 (192.168.163.135). SSH, déjà actif, est seulement vérifié.

Étape	Ce que fait le script
0. Nettoyage	Si un agent existe : arrêt, purge et suppression de `/var/ossec`.
1. Paquets	Mise à jour des listes de paquets.
2. Dépôt	Ajoute le dépôt Wazuh et sa clé GPG.
3. Agent	Installe `wazuh-agent` avec le manager 192.168.163.135, l'active et vérifie qu'il est `active`.
4. SSH	Vérifie que `ssh` est actif : c'est le service attaqué par force brute.
5. Apache	Installe et active `apache2`.

```
sudo WAZUH_MANAGER="192.168.163.135"
apt install -y wazuh-agent
sudo systemctl enable --now wazuh-
agent
sudo systemctl is-active wazuh-agent

sudo systemctl is-active ssh
sudo apt install -y apache2
```

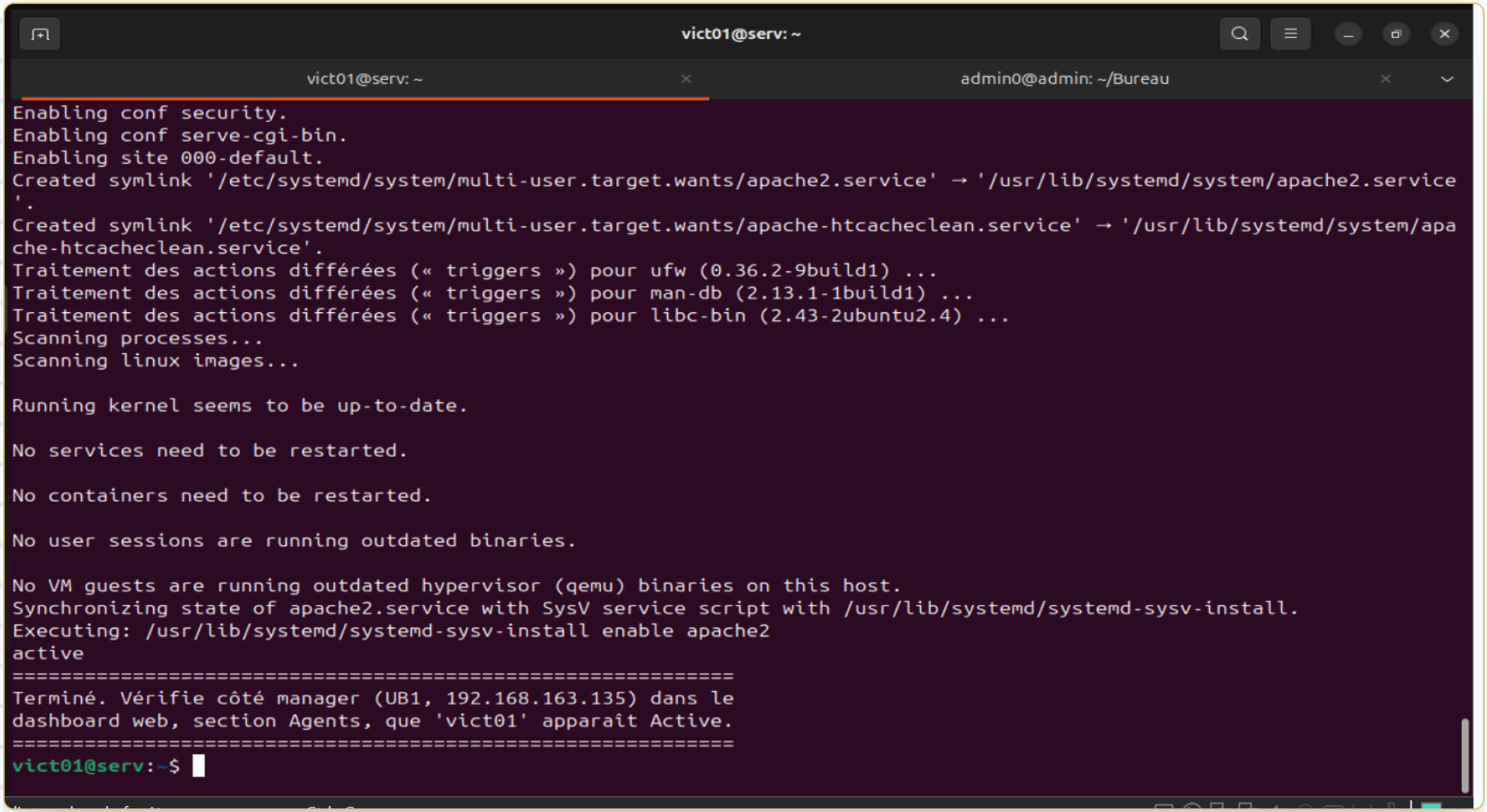
À retenir

Le script se termine en demandant de vérifier, dans la section Agents du dashboard, que la machine apparaît *Active*.

6.15 L'agent est déployé

SERV 5 oct. · 18:42

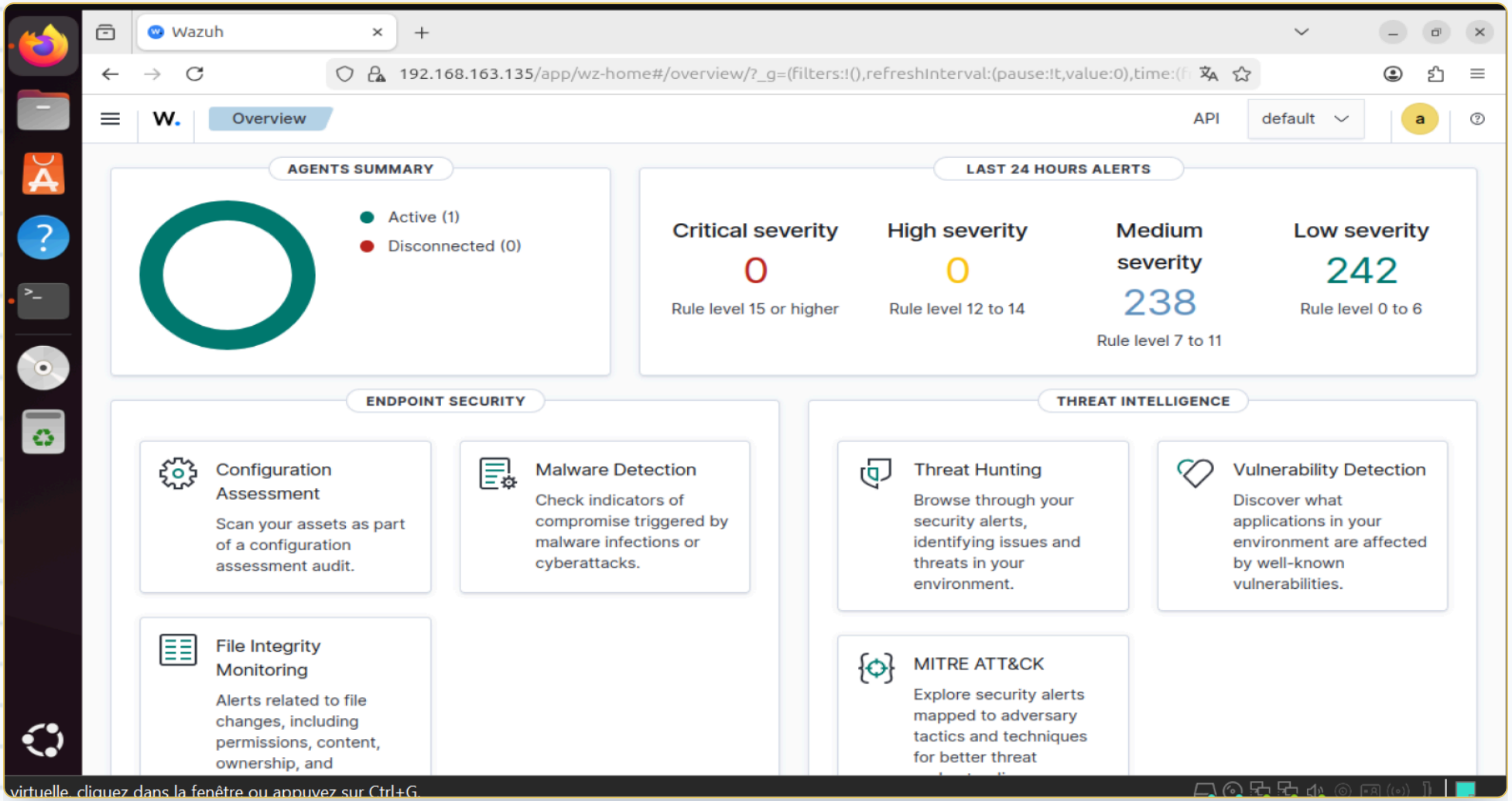
> Exécution du script 3 sur serv : à la fin, il invite à vérifier côté manager que l'agent apparaît *Active*.



6.16 La surveillance est confirmée

UB1 **5 oct. · 18:44**

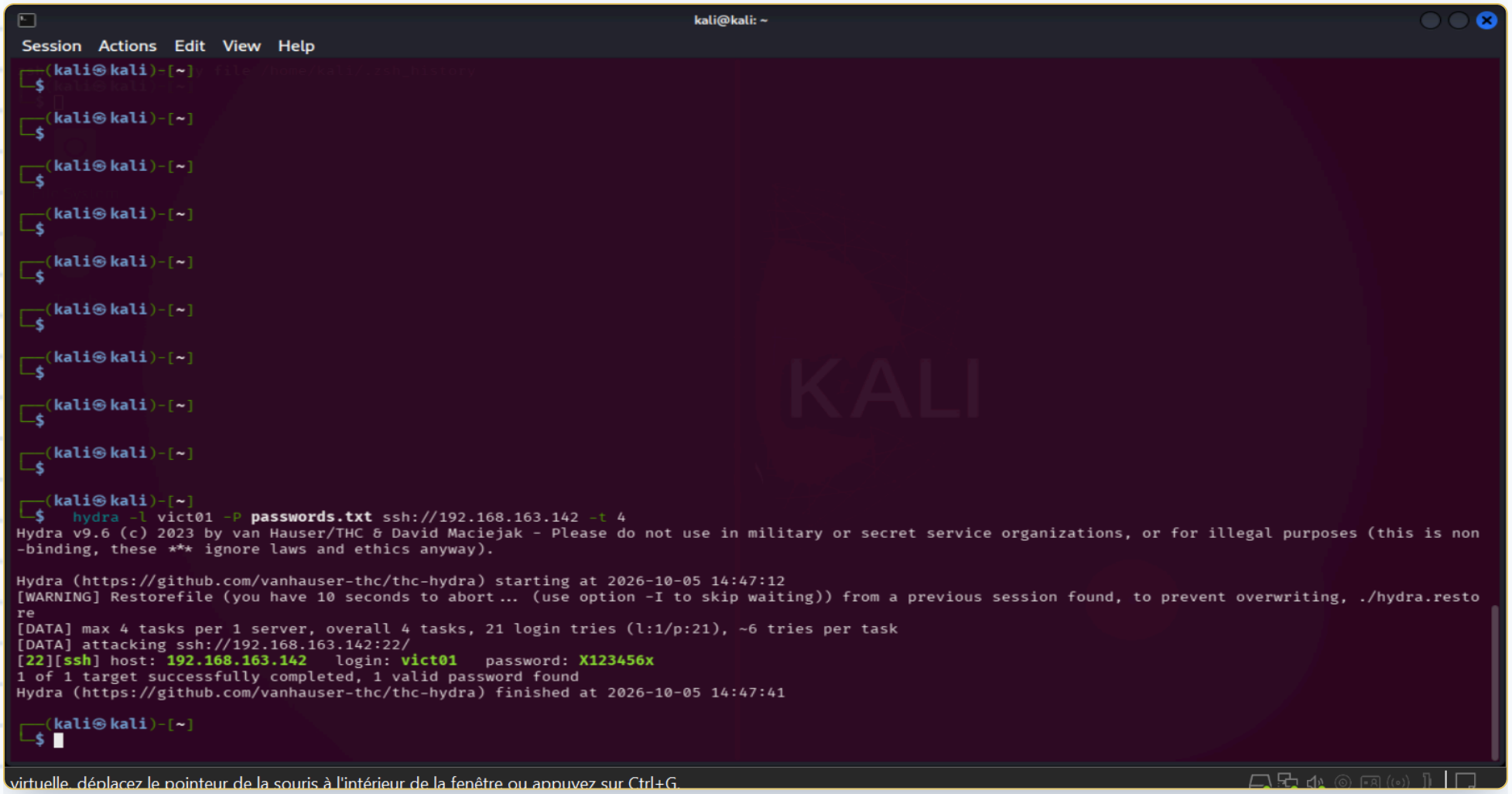
> Retour sur le dashboard : l'Agents Summary passe à Active (1).



6.17 L'attaque commence

KALI 5 oct. · 18:47

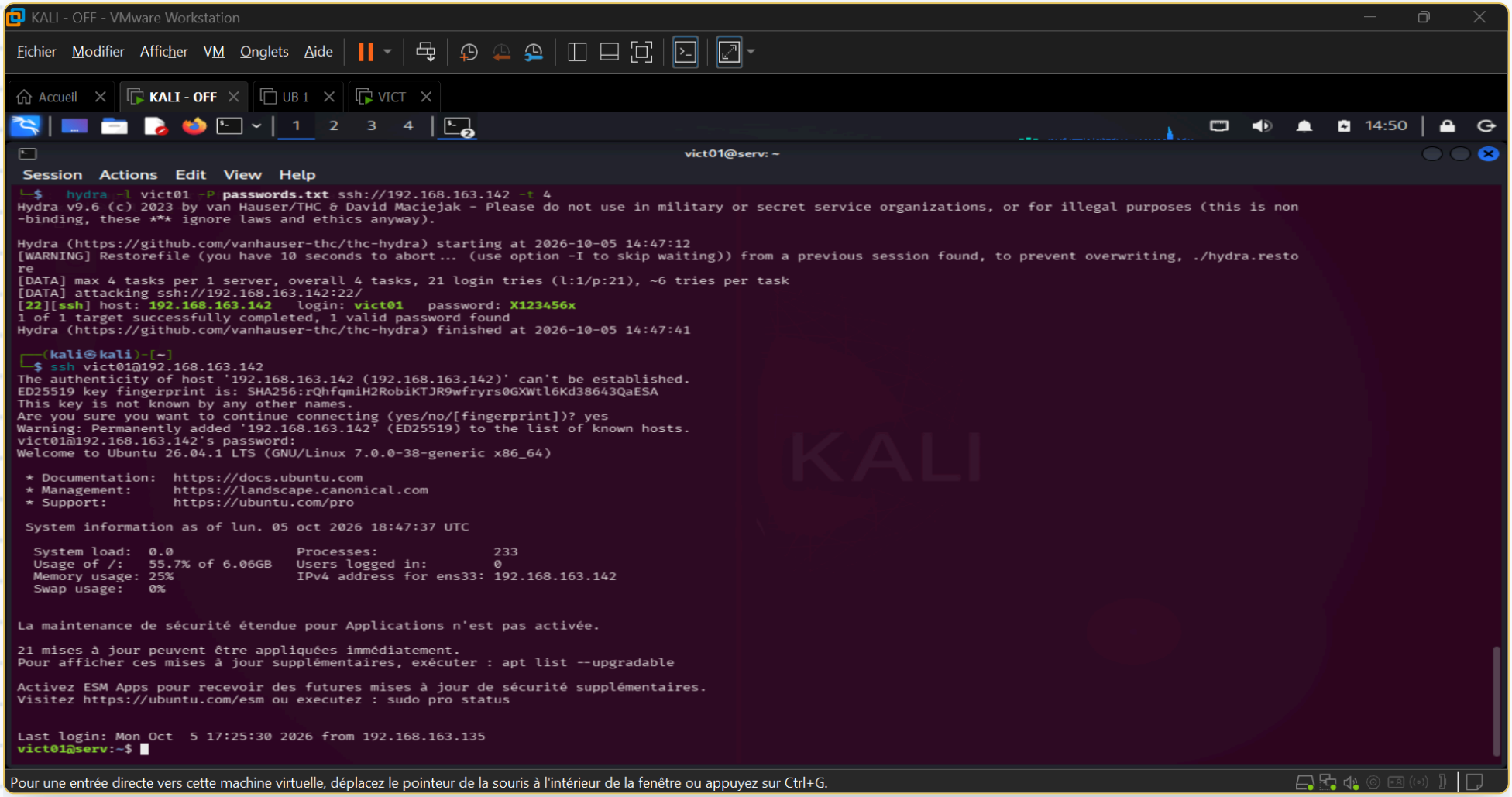
> Depuis Kali : `hydra` teste une liste de mots de passe contre le SSH de serv et la trouve en une tentative. Technique MITRE ATT&CK : **T1110** (Brute Force), tactique Accès initial.



6.18 L'accès est obtenu

KALI 5 oct. · 18:49

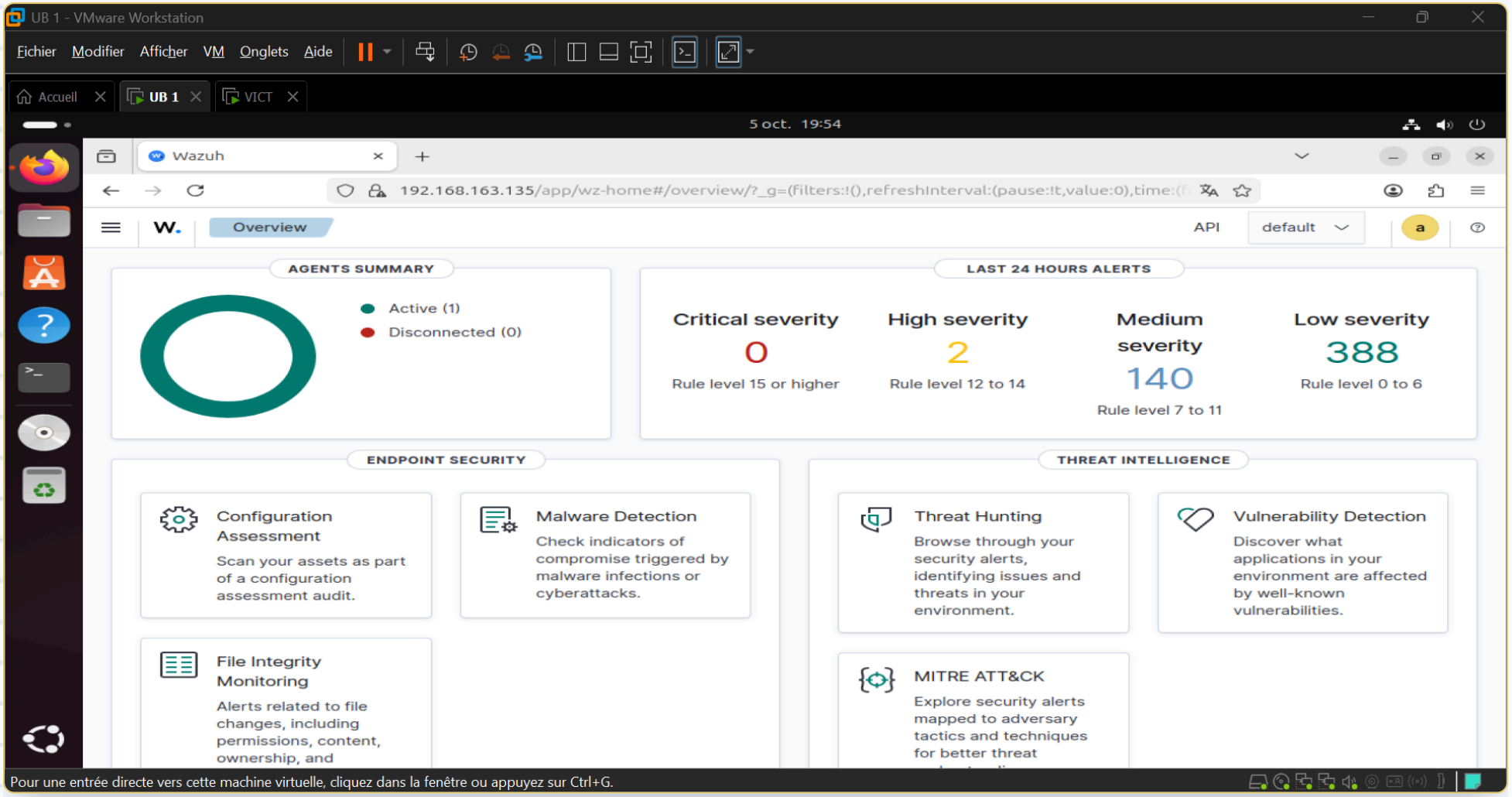
> Connexion SSH réussie avec le mot de passe compromis : l'attaquant est entré.



6.19 La première alerte remonte

UB1 5 oct. · 19:54

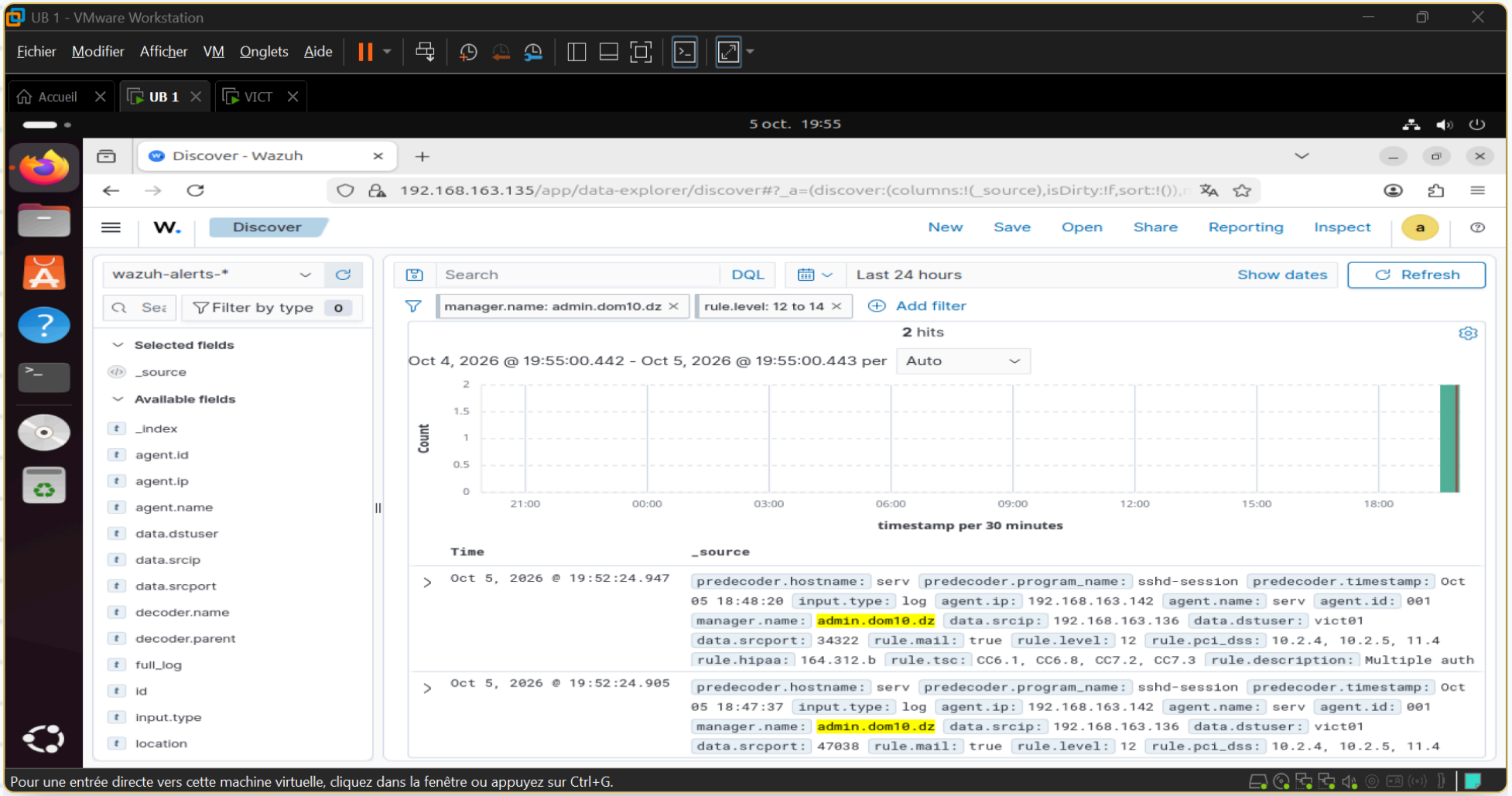
> L'Overview du dashboard affiche deux alertes de sévérité High (niveau 12 à 14) sur les dernières 24 heures.



6.20 Confirmation dans Discover

UB1 5 oct. · 19:55

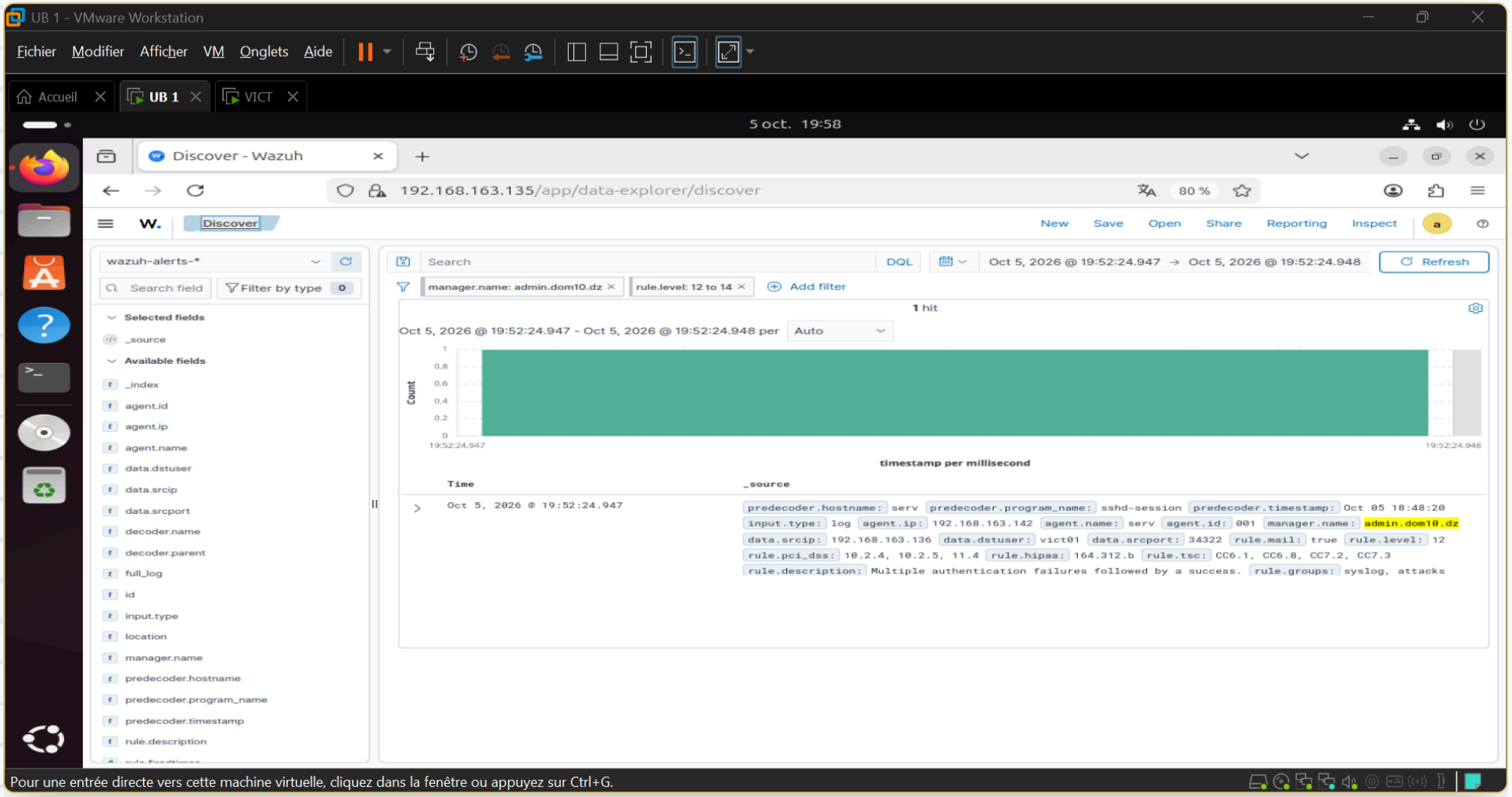
> Filtrage sur `rule.level: 12 to 14` : deux événements, dont l'enchaînement d'échecs suivi d'un succès.



6.21 L'événement, décortiqué

UB1 **5 oct. · 19:58**

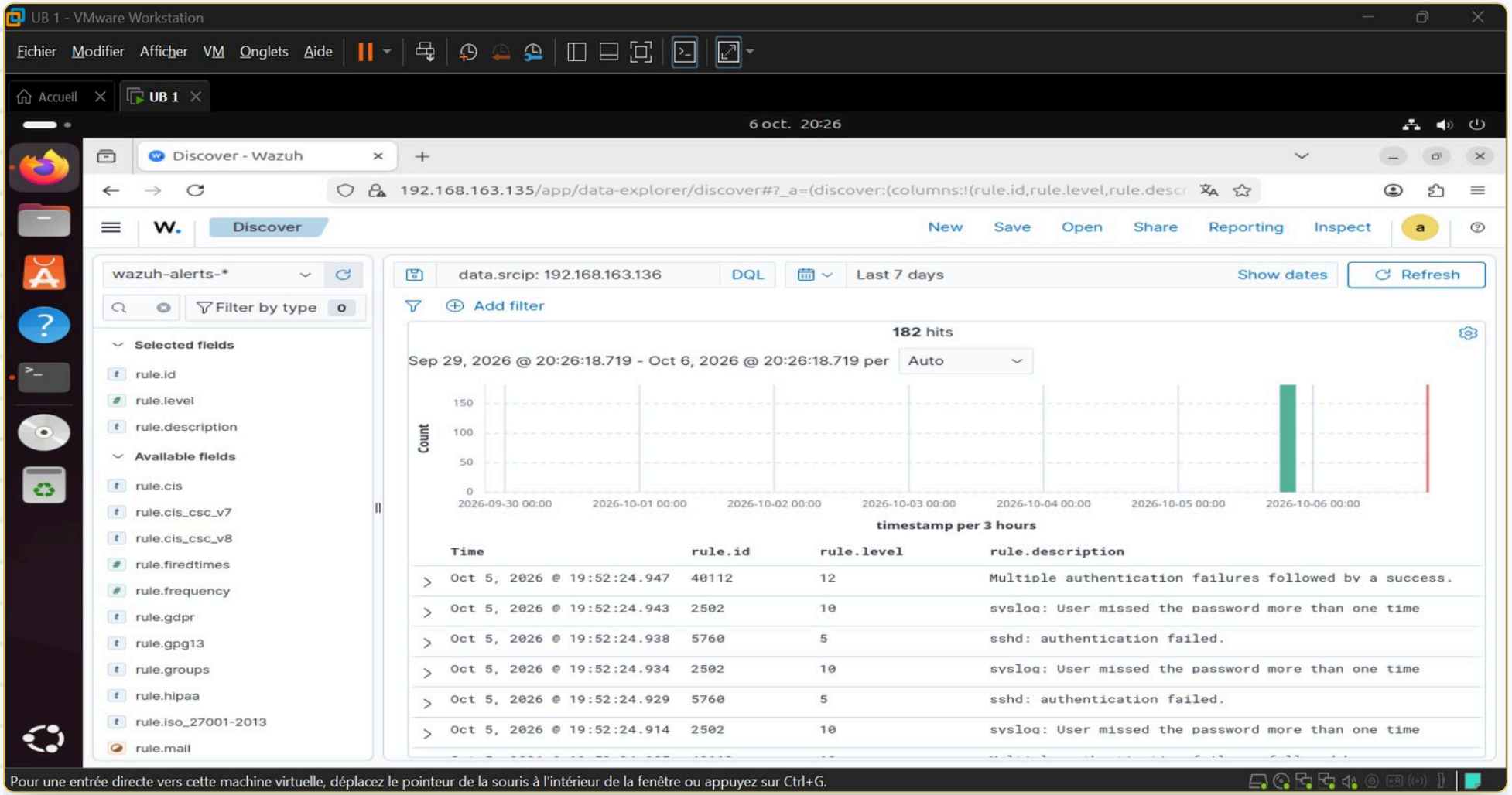
> Détail de l'alerte : « *Multiple authentication failures followed by a success* », avec son mapping PCI-DSS, HIPAA et TSC.



6.22 Les tentatives avant l'alerte

UB1 6 oct. · 20:26

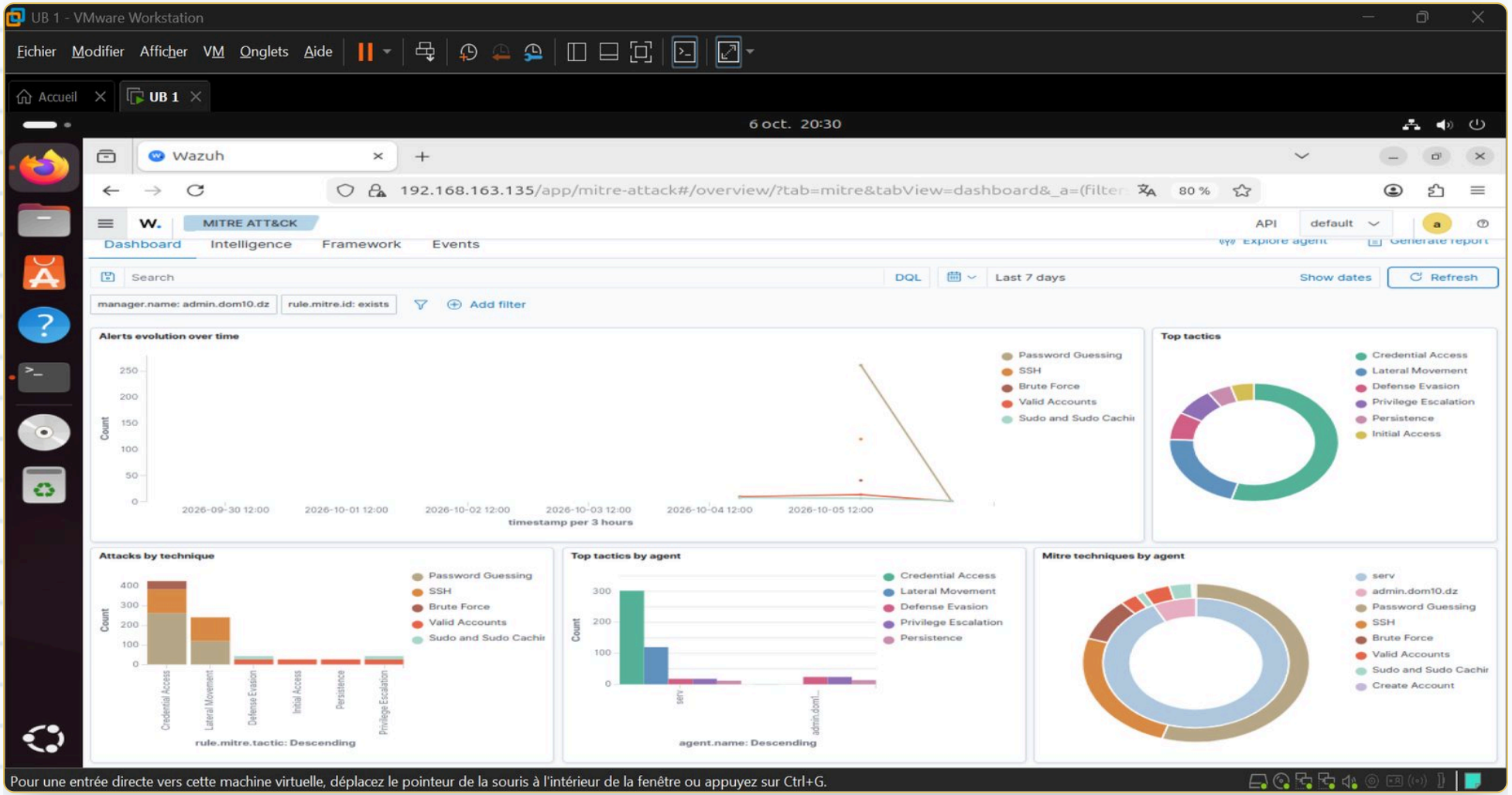
> Filtrée sur l'adresse de Kali, la recherche remonte 182 événements. Avant l'alerte de niveau 12, Wazuh a enregistré chaque échec : des échecs SSH isolés (règle 5760, niveau 5), puis le regroupement « mot de passe raté plusieurs fois » (règle 2502, niveau 10).



6.23 Les techniques MITRE ATT&CK observées

UB1 6 oct. · 20:30

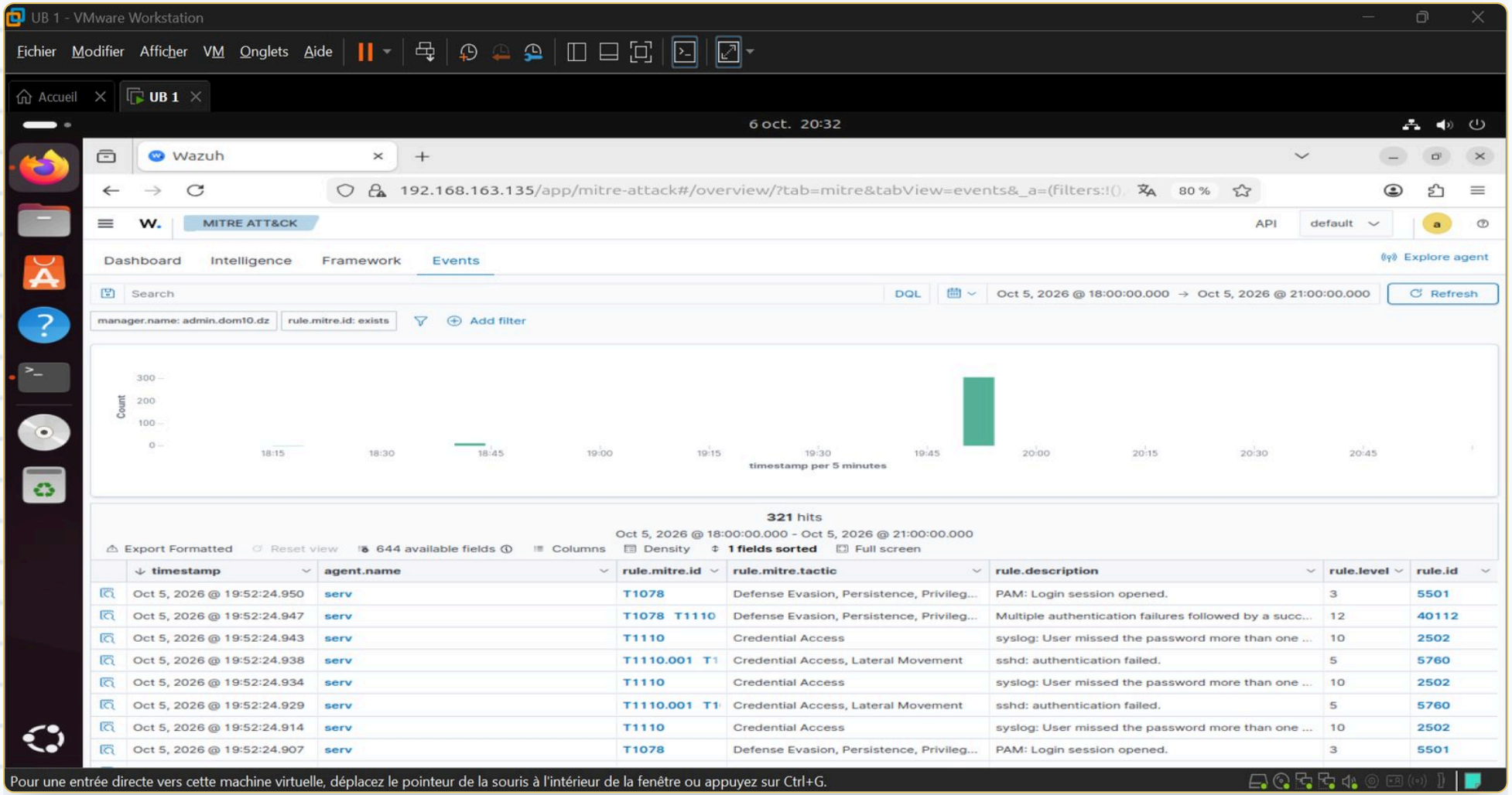
> Wazuh rattache les alertes à la matrice MITRE ATT&CK. Sur 7 jours, la tactique dominante est *Credential Access*, portée par Password Guessing, Brute Force et Valid Accounts : c'est ce que la règle 40112 annonçait (T1110 et T1078). Le SSH ressort aussi côté *Lateral Movement*.



6.24 Chaque alerte reliée à sa technique

UB1 6 oct. · 20:32

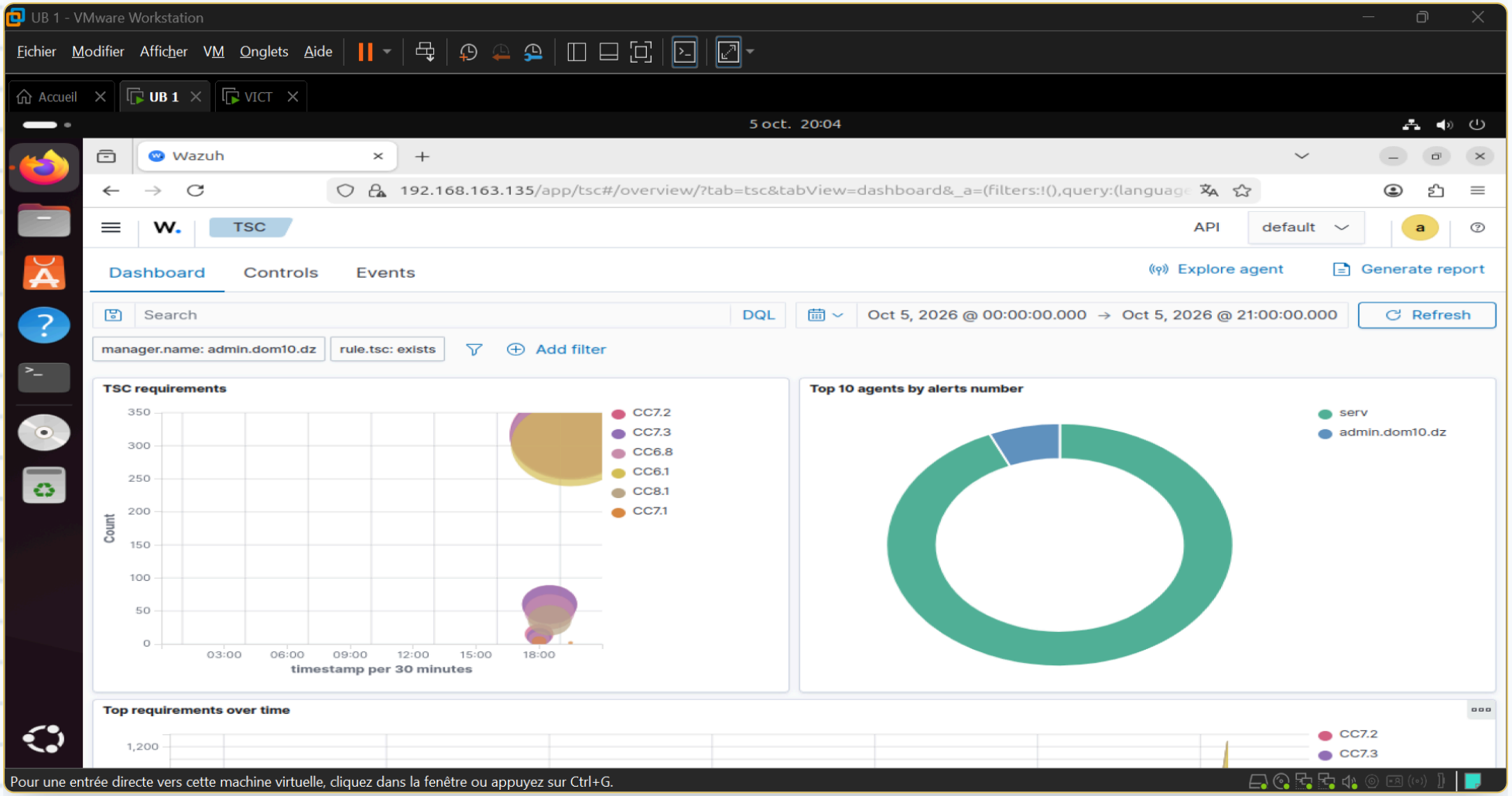
> L'onglet Events de MITRE ATT&CK détaille la fenêtre de l'attaque (321 événements, le 5 octobre entre 18 h et 21 h). L'échec SSH (règle 5760) relève de T1110.001, le regroupement d'échecs (règle 2502, niveau 10) de T1110, et l'alerte de niveau 12 (règle 40112) de T1078 et T1110.



6.25 La vue conformité

UB1 5 oct. · 20:04

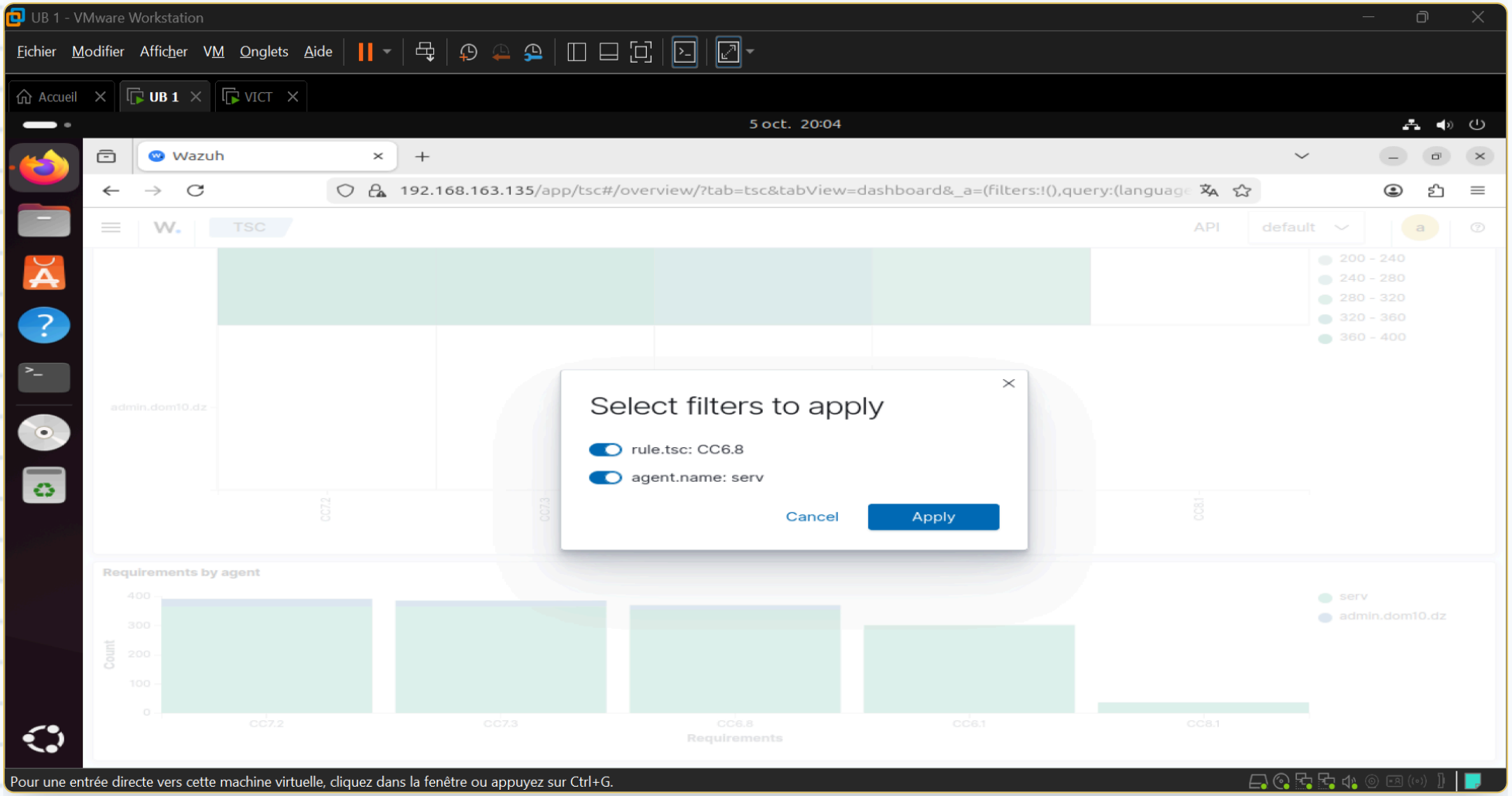
> Le tableau TSC montre la répartition des exigences déclenchées dans le temps et les agents les plus concernés.



6.26 Isoler l'incident

UB1 5 oct. · 20:04

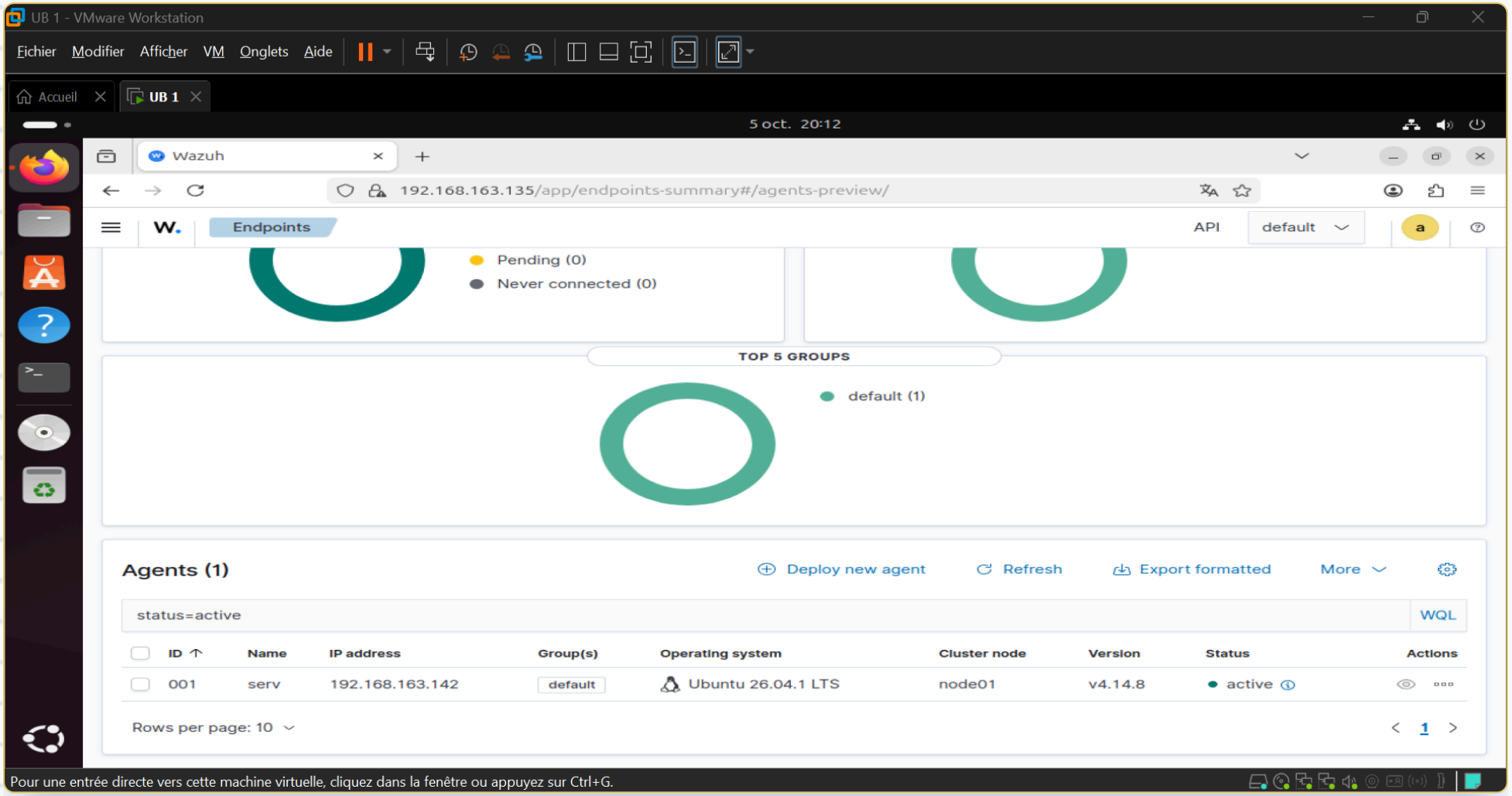
> Un filtre ciblé sur l'exigence `CC6.8` et l'agent `serv` permet de se concentrer sur cet incident précis.



6.27 Le bilan, côté endpoints

UB1 5 oct. · 20:12

> La liste des agents confirme : serv reste actif et surveillé, l'incident a été vu de bout en bout.



6.28 Ce que l'incident a révélé

> La règle déclenchée reconnaît un schéma classique de prise de pied initiale : plusieurs échecs d'authentification suivis d'un succès.

Règle déclenchée

Niveau 12 — *Multiple authentication failures followed by a success*

Conformité TSC

CC6.1 · CC6.8 · CC7.2 · CC7.3

PCI-DSS / HIPAA

10.2.4 · 10.2.5 · 11.4 · 164.312.b

Source → cible

192.168.163.136 (Kali) → vict01@192.168.163.142 (serv)

6.29 Bilan : théorie et laboratoire

Étape théorique (5.1)	Observé dans le labo
T+1 à T+3 : tentatives et écriture du journal	Hydra teste la liste de mots de passe ; chaque échec est écrit dans le journal d'authentification de la victime.
T+4 à T+6 : agent, serveur, corrélation	L'agent transmet les lignes ; le manager les décode et compte les échecs sur la fenêtre de temps.
T+7 à T+8 : connexion réussie, alerte	La connexion SSH aboutit ; la règle de niveau 12 se déclenche : alerte de haute sévérité créée.
T+9 et suivants : triage, investigation, réponse	Non rejoué ici : un seul opérateur, labo isolé (cf. 5.5, « Laboratoire contre SOC réel »).

Limite assumée

Le labo s'arrête à la détection : le triage (5.2), la réponse à incident (5.3) et le SOAR (5.4) restent, ici, des étapes décrites en théorie plutôt que rejouées en pratique.

Glossaire

Actif : ce qui a de la valeur. **Alerte** : signal ayant satisfait une règle. **Baseline** : comportement de référence. **CVE** : identifiant public d'une faille connue. **Corrélation** : lien entre plusieurs événements. **DMZ** : zone exposée mais isolée du réseau interne. **EDR** : détection et réponse sur les postes. **Éradication** : suppression de la cause d'un incident. **Événement** : log compris, avec ses champs. **Faux négatif** : attaque non détectée. **Faux positif** : alerte sans menace. **FIM** : surveillance de l'intégrité des fichiers. **Forensic** : analyse des preuves numériques. **IDS / IPS** : détection / prévention d'intrusion. **Incident** : menace confirmée. **IOC** : indicateur de compromission. **Log** : trace écrite par un système. **Mouvement latéral** : déplacement de machine en machine. **MITRE ATT&CK** : base des tactiques et techniques d'attaque. **MTTD / MTTR** : délai moyen de détection / de réponse. **NDR** : détection et réponse sur le réseau. **Normalisation** : noms de champs communs. **Observable** : élément à analyser (adresse, fichier). **Parsing** : découpage d'un log en champs. **PCAP** : fichier de capture de paquets. **Playbook** : procédure d'étapes pour un type d'incident. **Port SPAN** : port qui reçoit une copie du trafic. **Risque** : probabilité × impact. **SIEM** : centralisation, corrélation et alertes. **SOAR** : automatisation de la réponse. **SOC** : centre de supervision de la sécurité. **Syslog** : protocole d'envoi de journaux. **Threat hunting** : chasse proactive aux menaces. **TLS** : chiffrement du transport. **Triage** : tri et qualification des alertes. **TTP** : tactiques, techniques et procédures. **VLAN** : réseau local virtuel. **XDR** : détection étendue multi-sources.